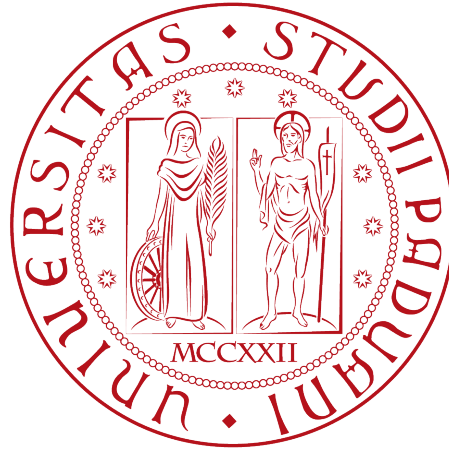


Università degli Studi di Padova

DIPARTIMENTO DI MATEMATICA “TULLIO LEVI-CIVITA”

CORSO DI LAUREA IN INFORMATICA



**Agenti AI per il CRM enterprise: analisi della
telemetria e automazione del testing**

Tesi di laurea triennale

Relatore

Prof. Francesco Ranzato

Laureando

Alessio Zanella

Matricola 2101079

ANNO ACCADEMICO 2025-2026

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

— Oscar Wilde

Dedicato a ...

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di stage, della durata di circa trecento ore, presso l'azienda Sia Informatica s.r.l.

Il progetto prevede l'integrazione di un sistema strutturato di testing automatizzato e telemetria all'interno dell'applicazione web *enterprise* Sia.Core, il cui core è composto da un client-side sviluppato in Angular e un server-side in .NET 9. L'approccio adottato si basa sull'utilizzo dell'intelligenza artificiale come elemento centrale del processo, con particolare riferimento a Spec-Kit, strumento che consente di definire in modo formale e strutturato le specifiche del lavoro svolto dall'AI, garantendo tracciabilità, riproducibilità e qualità nella generazione automatica del codice di test. Per la telemetria e l'osservabilità, il progetto adotta OpenTelemetry per la raccolta distribuita di metriche, trace e log, e Grafana come piattaforma di visualizzazione e analisi, al fine di valutare le prestazioni del sistema e identificare eventuali criticità in modo proattivo.

Gli obiettivi da raggiungere erano molteplici.

In primo luogo, è stato necessario approfondire le tecnologie richieste per lo sviluppo del progetto e comprendere l'applicativo Sia.Core, all'interno del quale la soluzione sarebbe stata integrata. In secondo luogo, si è resa necessaria l'analisi della strategia di testing più adatta al framework aziendale, insieme all'individuazione del modello di intelligenza artificiale più idoneo a supportare tale attività. Infine, il progetto è stato portato a termine attraverso l'integrazione della telemetria e l'implementazione dei test necessari al rilascio della soluzione in produzione.

“Life is really simple, but we insist on making it complicated”

— Confucius

Ringraziamenti

Innanzitutto, vorrei esprimere la mia gratitudine al Prof. Francesco Ranzato, relatore della mia tesi, per l'aiuto e il sostegno fornitomi durante la stesura del lavoro.

Desidero ringraziare con affetto i miei genitori per il sostegno, il grande aiuto e per essermi stati vicini in ogni momento durante gli anni di studio.

Ho desiderio di ringraziare poi i miei amici per tutti i bellissimi anni passati insieme e le mille avventure vissute.

Padova, Luglio 2026

Alessio Zanella

Indice

1	Introduzione	1
1.1	L'azienda	1
1.2	L'idea	2
1.3	Una metodologia di lavoro centrata sull'AI	2
1.4	Implementazioni realizzate	3
1.5	Organizzazione del testo	3
2	Descrizione dello stage	5
2.1	Introduzione al progetto	5
2.2	Requisiti e obiettivi	5
2.2.1	Obiettivi	5
2.2.2	Requisiti funzionali	6
2.2.3	Requisiti non funzionali	6
2.3	Analisi preventiva dei rischi	7
2.4	Pianificazione	8
2.4.1	Pianificazione settimanale	8
3	Analisi dei requisiti	12
3.1	Analisi del sistema	12
3.2	Casi d'uso	12
3.2.1	Attori	13
3.2.2	Elenco casi d'uso	13
3.3	Tracciamento dei requisiti	14
4	Tecnologie adottate	17
4.1	Studio e scelta delle tecnologie	17
4.2	Tecnologie e strumenti	17
5	Progettazione	21
5.1	Osservabilità nei sistemi distribuiti	21
5.2	Lo standard OpenTelemetry	22
5.3	Architettura della telemetria	22
5.3.1	Produzione dei dati di telemetria	22
5.3.2	Integrazione con OpenTelemetry Collector ed ecosistema Grafana	23
5.3.3	Scelte architetturali	24
5.4	Modello dei segnali e utilizzo degli attributi	25
5.4.1	Modello dei log	25

5.4.2	Modello delle tracce	26
5.4.3	Modello delle metriche	26
5.4.4	Tassonomia degli attributi	27
5.4.5	Tracciamento dell'identità utente	28
5.4.6	Exemplar: collegamento diretto da metrica a tracce	28
5.5	Design pattern applicati	29
5.5.1	Standardizzazione del counter errori controller tramite pattern Adapter	29
5.5.2	Pattern Adapter	30
5.5.3	Pattern Factory	31
5.6	Diagrammi delle classi del sistema di telemetria	32
5.6.1	Pipeline condiviso in <code>BaseApplicationBuilder</code>	32
5.6.2	Pattern <code>Sia{Modulo}Diagnostics</code> e Adapter	32
5.6.3	Arricchimento del contesto utente	32
6	Metodologia assistita da agenti AI	35
6.1	Contesto e motivazione	35
6.2	Ambiti di utilizzo	35
6.2.1	Studio delle tecnologie	35
6.2.2	Prove di implementazione	36
6.2.3	Propagazione della telemetria agli altri moduli	36
6.3	Workflow Agenti AI	36
6.4	Confronto pianificazione vs esecuzione	37
6.5	Limiti e mitigazioni	37
6.6	Model Context Protocol	38
6.6.1	Il problema che MCP risolve	38
6.6.2	Architettura	38
6.6.3	Le primitive del protocollo	39
6.6.4	Dall'integrazione al microservizio agentico	39
6.6.5	Sicurezza	40
6.6.6	Applicazione nel progetto	40
6.7	Considerazioni conclusive	41
7	Codifica	42
7.1	Organizzazione del codice	42
7.2	Pipeline <code>OpenTelemetry</code>	42
7.3	Classi <code>Sia{Modulo}Diagnostics</code>	43
7.4	Pattern <code>RecordMetricError</code> con Adapter	43
7.5	<i>Middleware</i> di arricchimento utente	44
7.6	Configurazione degli <i>exemplars</i>	44
7.7	<i>Log</i> Serilog con <i>sink</i> OTLP	45
7.8	Dashboard Grafana e versionamento	45
7.9	Telemetria delle performance hardware	46
7.9.1	Contesto e obiettivo	46
7.9.2	Scelte progettuali	47
7.9.3	Segnali prodotti	47
7.9.4	Dashboard Grafana	47

8 Conclusioni	49
8.1 Consuntivo finale	49
8.2 Raggiungimento degli obiettivi	49
8.3 Conoscenze acquisite	50
8.4 Retention e gestione dei dati di telemetria	50
8.5 Valutazione personale e sviluppi futuri	50
A Appendice A	52
Acronimi e abbreviazioni	53
Glossario	54
Bibliografia	56

Elenco delle figure

1.1	Logo Sia Informatica	1
3.1	Use Case - UC0: Scenario principale	13
4.1	Logo Angular	18
4.2	Logo .NET	18
4.3	Logo Spec-Kit	19
4.4	Logo OpenTelemetry	19
4.5	Logo Grafana	19
4.6	Logo Playwright	20
4.7	Logo Vitest	20
5.1	Architettura OTel Collector	24
5.2	Pipeline OpenTelemetry centralizzato in <code>BaseApplicationBuilder</code> : i tre provider (trace, metriche, log) condividono lo stesso <code>ResourceBuilder</code> e utilizzano <i>sampler</i> e <i>processor</i> custom per le <i>trace</i>	33
5.3	Struttura generica del pattern <code>Sia{Modulo}Diagnostics</code> con <code>ControllerDiagnosticsAdapter</code> annidato. Il controller invoca <code>RecordMetricError</code> del metodo condivi- so <code>BaseApiController</code> , passando il singleton <code>AsControllerDiagnostics</code> ; l'Adapter riconcilia il <code>Counter</code> per-modulo con il contratto comune <code>IControllerDiagnostics</code>	34
5.4	Flusso di arricchimento automatico delle <code>Activity</code> con l'identità utente. Il <i>middleware</i> pubblica <code>enduser.id</code> e <code>enduser.name</code> nel <code>Baggage</code> di OpenTelemetry; il <code>BaggageActivityProcessor</code> li copia come <i>tag</i> su ogni <code>Activity</code> creata, comprese quelle prodotte dalle classi <code>Sia{Modulo}Diagnostics</code>	34
7.1	Overview e durata richieste della dashboard di <i>Sia.Base</i>	45
7.2	Sezione richieste per <code>#{bucket}</code> della dashboard <i>Sia.Base</i>	46
7.3	Sezione trace e log della dashboard di <i>Sia.Grid</i>	46
7.4	Sezione risorse della dashboard di telemetria hardware	48

Elenco delle tabelle

3.1	Tabella del tracciamento dei requisiti funzionali	15
3.2	Tabella del tracciamento dei requisiti qualitativi	16
3.3	Tabella del tracciamento dei requisiti di vincolo	16
5.1	Resource attributes comuni a trace, metriche e log	27
5.2	Tag del modulo Sia.Base (operazioni Create, Read, Update, Delete (CRUD))	27
5.3	Tag degli span del modulo Sia.Auth	28
5.4	Tag degli span del modulo Sia.Grid	28
7.1	Span prodotte dal modulo <code>Sia.HealthChecks.Servers</code>	47
7.2	ObservableGauge del modulo <code>Sia.HealthChecks.Servers</code>	48

Capitolo 1

Introduzione

Il presente capitolo introduce il contesto aziendale in cui si è svolto lo stage, l'idea da cui il progetto ha preso forma, i contributi originali del lavoro presentato e l'organizzazione del documento.

1.1 L'azienda

Sia Informatica S.r.l. è una società con sede a Vigonza (PD) che, dal 1996, opera nel settore dello sviluppo software e della consulenza informatica per aziende di diversi comparti produttivi. Il *core business* consiste nella progettazione e realizzazione di soluzioni tecnologiche personalizzate, costruite su misura delle esigenze operative dei clienti. L'offerta comprende sistemi [Enterprise Resource Planning \(ERP\)](#) verticali, declinati per i settori del *packaging*, dell'arredamento e della moda con il marchio *Job3 ERP*, servizi web tra cui un [Customer Relationship Management \(CRM\)](#) (*Sales Connect*) e strumenti di gestione logistica, soluzioni di gestione documentale (fatturazione elettronica, firma digitale, conservazione sostitutiva), nonché tecnologie di integrazione avanzata come [Radio-Frequency Identification \(RFID\)](#) e un assistente basato sull'intelligenza artificiale denominato *Kitsune*.

La filosofia aziendale si ispira al principio giapponese del *Kintsugi*, l'arte di valorizzare le imperfezioni trasformandole in punti di forza: in modo analogo, Sia Informatica non si limita a fornire strumenti software isolati, ma costruisce connessioni profonde tra i processi aziendali, con l'obiettivo di armonizzare e ottimizzare i flussi operativi. Con oltre vent'anni di esperienza, l'azienda è oggi un partner tecnologico consolidato per le imprese del Nord-Est italiano che cercano soluzioni integrate, scalabili e sostenibili nel tempo.



Figura 1.1: Logo Sia Informatica

1.2 L'idea

Tutti i prodotti aziendali condividono una stessa base tecnologica, *Sia.Core*: un'applicazione *web enterprise* che cresce di funzionalità mese dopo mese, distribuita su decine di installazioni di clienti diversi. Una piattaforma di queste dimensioni vive di un compromesso delicato: ogni nuova funzionalità deve poter essere introdotta rapidamente, ma senza intaccare la stabilità di ciò che già funziona.

A questo compromesso si aggiunge una difficoltà tipica delle applicazioni distribuite: quando in produzione qualcosa non va come previsto, capire cosa sta accadendo e perché non è affatto immediato. Un malfunzionamento può avere origine in un punto del sistema e manifestarsi in un punto completamente diverso, con un ritardo difficile da imputare. Senza strumenti adatti, lo sviluppatore si trova a indagare a posteriori, con informazioni frammentarie e dipendendo dalla memoria di chi ha installato il software.

L'idea alla base dello stage nasce da queste due esigenze parallele:

- dare al team di sviluppo una rete di sicurezza che permetta di intervenire sul codice del framework con la confidenza che nulla di già funzionante venga rotto, ovvero un sistema di *testing* automatico integrato e ripetibile;
- dare a chi si occupa di assistenza e diagnosi uno strumento che renda visibile il comportamento dell'applicazione in tempo reale, ovvero un sistema di telemetria che osservi in modo sistematico ciò che il software sta facendo e ne registri le prestazioni e le anomalie.

Il presente documento è dedicato in modo prevalente al secondo filone, mentre del primo viene data una panoramica orientata alla metodologia adottata e ai risultati ottenuti.

1.3 Una metodologia di lavoro centrata sull'AI

Un elemento che ha attraversato trasversalmente entrambe le linee di lavoro è stato l'utilizzo di un agente di intelligenza artificiale come *pair engineer* strutturato. La creazione e lo sviluppo di più agenti di codifica all'interno di un *framework* di queste dimensioni non è stata banale: per evitare che generasse codice incoerente o che inventasse pezzi di realtà non esistenti ("allucinazioni"), si è dovuta progettare diversi *tools*, tra i quali:

- nuovi agenti AI specializzati, configurati per operare su specifici moduli del sistema e guidati tramite prompt strutturati e contesto controllato, con convenzioni del *framework* in modo che la propagazione della telemetria e dei test ai vari moduli avvenga in modo uniforme;
- un insieme di *skill* procedurali: piccoli regolamenti scritti per l'agente, per la specializzazione degli agenti in determinati compiti, come: lo sviluppo dei vari segnali di telemetria, la creazione di dashboard per la visualizzazione di dati, la scrittura di test di vario tipo, ecc.;

- l'uso del [Model Context Protocol \(MCP\)](#), uno standard recente che consente all'agente di interrogare direttamente i sistemi coinvolti (*Grafana*, il database, gli strumenti di test) anziché lavorare su assunzioni potenzialmente errate.

1.4 Implementazioni realizzate

Il lavoro svolto durante il periodo di stage ha prodotto diversi contributi originali, sintetizzabili come segue:

- la **progettazione completa di un'architettura di osservabilità** per *Sia.Core*, dalla scelta dello stack (*OpenTelemetry* + *Grafana LGTM*) all'integrazione con l'infrastruttura esistente, passando alla standardizzazione del *pipeline*, in modo che ogni nuovo modulo possa aderirvi senza conoscerne i dettagli, terminando con la presentazione dei dati raccolti in dashboard apposite.
- la definizione di una **tassonomia coerente** di metriche, *trace* e attributi per i moduli centrali del *framework* (autenticazione, singoli componenti *Angular* come griglie e form, prestazioni dell'hardware, interrogazione del database, ecc.), pensata per evitare l'esplosione della cardinalità lato Prometheus e per consentire il confronto omogeneo tra moduli e clienti diversi;
- l'introduzione di due meccanismi per il **miglioramento dell'analisi della telemetria**: l'arricchimento automatico di ogni *trace* con l'identità dell'utente autenticato e l'uso degli *exemplar* per collegare metriche aggregate alla *trace* che le ha prodotte, che rendono possibile una catena diagnostica completa: dal grafico Grafana alla richiesta specifica, passando per l'utente coinvolto e i *log* emessi nel suo contesto;
- l'introduzione, da zero, di un'**infrastruttura di testing automatico** nel *framework* backend, articolata su *unit test* per la logica di business e *integration test* per la verifica dell'attendibilità dei dati presenti nei database, delle procedure SQL utilizzate e delle prospettive presenti per descrivere la struttura dei vari componenti;
- lo sviluppo di **test anche lato frontend**, comprendenti sia *unit test* tradizionali per la verifica della logica applicativa, sia test di validazione *end-to-end*, finalizzati a verificare il corretto comportamento e la corretta visualizzazione dell'applicazione dal punto di vista dell'utente finale;
- come già approfondito nella sezione precedente (1.3), la **creazione di agenti AI e skill** per la replicabilità e la propagazione di telemetria e test a contesti di sviluppo analoghi nel *framework*.

1.5 Organizzazione del testo

Il [secondo capitolo](#) descrive gli obiettivi del periodo di stage, la pianificazione del lavoro e l'analisi preventiva dei rischi.

Il terzo capitolo approfondisce le funzionalità del prodotto, presentando i requisiti e i casi d'uso individuati.

Il quarto capitolo presenta le tecnologie e gli strumenti adottati durante lo stage, soffermandosi sui criteri di scelta.

Il quinto capitolo illustra le scelte progettuali e architetture del sistema di telemetria, con rinvio in appendice ai dettagli tecnici di ampia dimensione.

Il sesto capitolo descrive in dettaglio la metodologia di lavoro AI-assistita, le *skill* sviluppate, il MCP e le sue applicazioni pratiche al progetto.

Il settimo capitolo espone gli aspetti implementativi salienti, mantenendo nel corpo principale solo le descrizioni di alto livello e relegando in appendice i frammenti di codice e i dettagli più estesi.

L'ottavo capitolo riassume i risultati raggiunti, le conoscenze acquisite e gli sviluppi futuri più promettenti.

Riguardo alla stesura del testo, sono state adottate le seguenti convenzioni tipografiche:

- gli acronimi, le abbreviazioni e i termini ambigui o di uso non comune sono definiti nel glossario, situato alla fine del presente documento;
- alla prima occorrenza di un termine riportato nel glossario è utilizzata la nomenclatura “parola (abbreviazione)”, mentre nelle occorrenze successive si utilizza solamente l'abbreviazione;
- i termini in lingua straniera o appartenenti al gergo tecnico sono evidenziati con il carattere *corsivo*.

Capitolo 2

Descrizione dello stage

Il presente capitolo descrive gli obiettivi del progetto di stage, la pianificazione delle attività in termini di ore di lavoro e l'analisi preventiva dei principali rischi che potrebbero insorgere nel corso del suo svolgimento.

2.1 Introduzione al progetto

Nel contesto dello sviluppo software moderno, le applicazioni tendono ad assumere un'architettura sempre più distribuita e complessa. La capacità di monitorare il comportamento di tali sistemi in tempo reale, diagnosticare anomalie e analizzare le prestazioni costituisce un requisito fondamentale per garantire la qualità del servizio. Il progetto *Sia.Web* ha richiesto la progettazione e l'implementazione di un sistema di telemetria completo, in grado di raccogliere e correlare *trace*, metriche e *log* per offrire una visione unificata del comportamento delle applicazioni in produzione. I fondamenti teorici dell'osservabilità sono trattati in dettaglio nel capitolo 5.

2.2 Requisiti e obiettivi

2.2.1 Obiettivi

1. Identificare le tecnologie più adatte allo sviluppo del progetto e i moduli più importanti da tracciare tramite telemetria.
2. Analizzare i requisiti di osservabilità dell'ecosistema *Sia.Web* e identificare le soluzioni tecnologiche adeguate.
3. Progettare un'architettura di telemetria basata sullo standard [OpenTelemetry \(OTel\)](#), integrando il framework con le tecnologie .NET esistenti.
4. Definire le strategie di campionamento delle trace e di gestione delle metriche in ottica di scalabilità verso ambienti di produzione.
5. Implementare gli strumenti di strumentazione per i moduli applicativi principali (autenticazione, accesso ai dati, griglia).

6. Configurare la piattaforma di visualizzazione Grafana e i relativi backend di storage.

2.2.2 Requisiti funzionali

- RF1. Il sistema deve tracciare il percorso completo di ogni richiesta HTTP in ingresso, incluse le operazioni di accesso al database.
- RF2. Le operazioni CRUD verso il database devono essere riconoscibili tramite attributi che espongono il metodo e la repository da cui è stata scatenata la richiesta.
- RF3. Il sistema deve esporre metriche operative per i moduli di autenticazione, accesso ai dati e gestione della griglia.
- RF4. I log devono essere correlabili alle trace tramite `TraceId` e `SpanId`.
- RF5. Il sistema deve essere attivabile e disattivabile tramite configurazione, senza richiedere modifiche al codice.
- RF6. Deve essere possibile configurare indipendentemente il campionamento per ciascuna sorgente di trace.
- RF7. Deve essere possibile visualizzare i valori di trace e metriche raggruppate in dashboard Grafana.
- RF8. Le dashboard di Grafana devono raccogliere i dati da più servizi e installazioni diverse di ciascun cliente, e permettere di filtrare per gli stessi.
- RF9. Sviluppo di *Claude skill* per permettere la propagazione della telemetria in tutto il sistema.
- RF10. Il sistema deve arricchire automaticamente ogni span con l'identità dell'utente autenticato (`enduser.id`, `enduser.name`) senza richiedere modifiche ai singoli moduli applicativi.
- RF11. Le metriche di tipo *Histogram* devono esporre *exemplars*, ovvero puntatori al `TraceId` del campione, per abilitare la navigazione dalla metrica aggregata alla trace corrispondente.

2.2.3 Requisiti non funzionali

- RNF1. La telemetria non deve degradare sensibilmente le prestazioni in produzione.
- RNF2. Le metriche devono essere progettate per non generare un numero eccessivo di serie temporali in Prometheus; in particolare, i tag ad alta variabilità (es. identificativi utente o di record) non devono essere usati come attributi di metriche.
- RNF3. I segnali prodotti devono essere compatibili con qualsiasi backend [OTel](#)-compatibile.
- RNF4. Le impostazioni devono essere modificabili per ambiente (sviluppo, *staging*, produzione) tramite *appsettings*.

2.3 Analisi preventiva dei rischi

Durante la fase iniziale di analisi, sono stati individuati i principali rischi che sarebbero potuti emergere nel corso dello stage. Per ciascun rischio identificato, è stata definita una possibile strategia di mitigazione, al fine di ridurre l'impatto e garantire il corretto avanzamento delle attività previste.

1. Tecnologie sconosciute

Descrizione: alcune delle tecnologie previste nel progetto risultano totalmente o parzialmente sconosciute, con il rischio di rallentare le attività nelle fasi iniziali dello stage.

Soluzione: pianificazione di un'attività strutturata di studio e approfondimento, attraverso l'utilizzo di documentazione ufficiale, tutorial online, corsi forniti dall'azienda e lo sviluppo di piccoli progetti di consolidamento, al fine di acquisire rapidamente le competenze necessarie.

2. Gestione del tempo

Descrizione: la pianificazione delle attività potrebbe risultare non accurata, soprattutto a causa della presenza di tecnologie nuove e della componente sperimentale legata all'uso dell'intelligenza artificiale, con il rischio di ritardi rispetto alle scadenze previste.

Soluzione: adottare il modello agile [SCRUM](#) per definire obiettivi intermedi settimanali, monitoraggio continuo dello stato di avanzamento e revisione periodica della pianificazione, in modo da adattare le attività in base alle difficoltà riscontrate.

3. Allucinazioni [Large Language Model \(LLM\)](#)

Descrizione: i modelli di intelligenza artificiale utilizzati per la generazione automatica di test possono produrre risultati non corretti, incoerenti o non aderenti alle specifiche (fenomeno noto come "allucinazione"), compromettendo l'affidabilità del codice generato.

Soluzione: adozione di tecniche di verifica e validazione dei risultati prodotti, affiancate da un'attenta progettazione dei *prompt* e dall'utilizzo di specifiche strutturate. Inoltre, iterazioni successive e revisioni manuali consentono di migliorare progressivamente la qualità dell'output generato.

4. Integrazione tra tecnologie

Descrizione: l'integrazione tra strumenti diversi, quali Spec-Kit, OpenTelemetry e Grafana, può presentare difficoltà legate a compatibilità e interfacciabilità, con il rischio di rallentare lo sviluppo o compromettere il corretto flusso dei dati.

Soluzione: utilizzo di un approccio incrementale all'integrazione, partendo da configurazioni di base e aumentando progressivamente la complessità. Consultazione della documentazione, utilizzo di ambienti di sviluppo locali per test e validazione, e confronto con il team aziendale per la risoluzione di eventuali criticità.

5. Qualità dei test generati automaticamente

Descrizione: i test generati tramite modelli di intelligenza artificiale potrebbero risultare incompleti, non corretti o con copertura insufficiente, compromettendo l'efficacia del processo di *testing*.

Soluzione: introduzione di attività di validazione e revisione dei test generati, utilizzo di metriche di copertura del codice e iterazione sui *prompt* per migliorare progressivamente la qualità dei risultati.

6. Differenze di comportamento tra ambiente di sviluppo e produzione

Descrizione: il sistema potrebbe presentare comportamenti differenti tra ambiente di sviluppo, test e produzione, a causa di differenze nella configurazione, nei dati o nell'infrastruttura. Ciò può portare a malfunzionamenti non rilevati durante le fasi di sviluppo e *testing*.

Soluzione: utilizzo di ambienti di *staging* il più possibile allineati alla produzione, esecuzione di test approfonditi prima del rilascio e monitoraggio continuo tramite strumenti di telemetria, al fine di individuare e risolvere tempestivamente eventuali anomalie.

7. Overfitting dei prompt per modelli LLM

Descrizione: i *prompt* progettati per la generazione automatica dei test potrebbero risultare troppo specifici rispetto ai casi inizialmente considerati, adattandosi eccessivamente al codice analizzato nelle prime fasi. Questo fenomeno può portare a una ridotta capacità di generalizzazione, con conseguente inefficacia nella generazione di test corretti o completi per nuove porzioni di codice introdotte successivamente.

Soluzione: progettazione dei *prompt* in modo generale e modulare, evitando eccessiva specializzazione su casi specifici. Introduzione di test su scenari diversi e codice non visto durante le fasi iniziali, accompagnata da iterazioni e revisioni periodiche dei *prompt*, al fine di migliorarne la capacità di generalizzazione.

2.4 Pianificazione

L'attività di stage è stata pianificata su una durata complessiva di 8 settimane, per un totale di 300 ore.

La pianificazione è stata strutturata in modo progressivo, suddividendo il lavoro in fasi settimanali, ciascuna con obiettivi specifici e attività mirate, al fine di garantire un apprendimento graduale e un'integrazione efficace delle tecnologie coinvolte.

2.4.1 Pianificazione settimanale

2.4.1.1 Settimane 1 e 2: Studio dell'architettura del sistema

Le prime due settimane sono state dedicate all'analisi dell'architettura dell'applicazione Sia.Core, con particolare attenzione alla separazione tra componente *client-side* e *server-side*.

- Analisi della struttura del progetto *Angular*: moduli, componenti, servizi, *routing*
- Studio delle convenzioni e pattern architetturali adottati (*lazy loading*, *state management*)
- Revisione del codice esistente con supporto del team di sviluppo

- Analisi della struttura del progetto *.NET 9*: *layer*, *Application Program Interface (API)*, *middleware*, *dependency injection*
- Studio dei pattern architetturali utilizzati (*Clean Architecture*, *CQRS*, *repository pattern*)
- Comprensione dei flussi di dati tra *client* e *server*: *API REST*, contratti, autenticazione
- Mappatura dei componenti critici candidati all'integrazione di test e telemetria

2.4.1.2 Settimana 3: Studio dell'architettura del sistema

La terza settimana è stata dedicata all'apprendimento degli strumenti chiave del progetto: *Spec-Kit*, *OpenTelemetry* e *Grafana*.

- *Spec-Kit*: studio della documentazione, comprensione del modello di specifica *AI-driven*, esercitazioni pratiche
- *OpenTelemetry*: studio dei concetti fondamentali (*traces*, *metrics*, *logs*), integrazione con *Angular* e *.NET*
- *Grafana*: configurazione istanza locale, connessione alle sorgenti dati *OpenTelemetry*, dashboard di base
- Realizzazione di un ambiente di sviluppo locale integrato con tutti e tre gli strumenti

2.4.1.3 Settimana 4: Strategia di testing e selezione del modello AI

Durante la quarta settimana è stata definita la strategia di *testing* da adottare, identificando le principali tipologie di test (unitari, di integrazione, *end-to-end* e di performance).

Parallelamente, è stata effettuata un'analisi comparativa tra diversi modelli di intelligenza artificiale, con l'obiettivo di selezionare quello più adatto alla generazione automatica di test in integrazione con *Spec-Kit*.

- Identificazione delle tipologie di test: *unit test*, *integration test*, *E2E test*, *performance test*
- Valutazione comparativa dei modelli *Artificial Intelligence (AI)* disponibili (*GPT-4o*, *Claude*, *Gemini*) per qualità e integrazione con *Spec-Kit*
- Selezione e motivazione del modello *AI* adottato
- Stesura del documento di strategia condiviso con il team

2.4.1.4 Settimana 5: Prompt engineering e prima generazione di test

La quinta settimana è stata dedicata allo studio delle tecniche di *prompt engineering* e alla prima generazione di test tramite intelligenza artificiale.

- Studio delle tecniche di *prompt engineering*: *chain-of-thought*, *few-shot*, *context injection*
- Definizione delle specifiche tramite *Spec-Kit* per i componenti individuati nella settimana 4
- Stesura dei *prompt* per la generazione di test lato *client* (*Angular*) e lato *server* (*.NET 9*)
- Prima esecuzione dei *prompt* sul modello [AI](#) selezionato
- Raccolta, analisi e valutazione del codice generato

2.4.1.5 Settimana 6: Iterazione sui test e prima integrazione della telemetria

Nella sesta settimana il lavoro si è concentrato sul miglioramento iterativo dei test generati e sull'integrazione della telemetria nel sistema.

- Analisi dei risultati: identificazione di test mancanti, errati o incompleti
- Raffinamento dei *prompt* e nuove iterazioni di generazione
- Integrazione di *OpenTelemetry* nei componenti server-side: trace e metrics sulle [API](#) principali
- Prima configurazione delle dashboard *Grafana* per la visualizzazione dei dati
- Esecuzione dei test e verifica dei risultati su ambiente di sviluppo
- Documentazione delle iterazioni e delle decisioni prese

2.4.1.6 Settimana 7: Validazione del sistema integrato e tuning delle dashboard

La settima settimana ha previsto l'esecuzione di una campagna completa di test e la validazione dell'intero sistema integrato.

- Esecuzione della suite completa di test su ambiente di *staging*
- Analisi della copertura del codice e identificazione delle aree scoperte
- Verifica della telemetria *end-to-end*: dall'evento applicativo alla dashboard *Grafana*
- Ottimizzazione delle dashboard *Grafana*: alert, soglie, visualizzazioni avanzate
- Correzione dei test falliti e ulteriore iterazione sui *prompt* se necessario
- Stesura del report di qualità intermedio

2.4.1.7 Settimana 8: Messa in produzione e documentazione finale

L'ultima settimana è stata dedicata alla messa in produzione della soluzione e alla redazione della documentazione finale.

- Preparazione e revisione finale dell'ambiente di produzione
- Deploy della configurazione *OpenTelemetry* e *Grafana* in produzione
- Stesura della documentazione tecnica finale (architettura, *prompt*, modello [AI](#), risultati)
- Stesura della relazione di stage con analisi critica del lavoro svolto
- Presentazione finale dei risultati al team aziendale

Capitolo 3

Analisi dei requisiti

Il presente capitolo descrive le funzionalità del prodotto, formalizzate sotto forma di requisiti di progetto, e relativi casi d'uso associati.

3.1 Analisi del sistema

Il prodotto sviluppato durante il periodo di stage mette a disposizione dello sviluppatore un sistema che permette la creazione automatizzata di test e il monitoraggio dello stato del sistema *Sia.Core* tramite dashboard.

Dal punto di vista funzionale, la soluzione si compone di due macro-sottosistemi:

- un sottosistema di *testing AI-driven*, basato sull'integrazione tra *Spec-Kit* e un modello [LLM](#), che guida la generazione strutturata di test unitari e di integrazione a partire da specifiche formali;
- un sottosistema di *osservabilità*, basato sullo standard [OTel](#) e sullo stack *Grafana* (Tempo, Prometheus, Loki), che raccoglie trace, metriche e log prodotti dall'applicazione e ne permette la visualizzazione e la correlazione tramite dashboard.

I due sottosistemi sono indipendenti sul piano tecnologico ma complementari sul piano metodologico: la telemetria fornisce il feedback che guida il miglioramento iterativo dei test, mentre i test supportano la validazione continua dell'strumentazione.

3.2 Casi d'uso

La presente sezione illustra i casi d'uso analizzati, corredati dai relativi diagrammi. I diagrammi dei casi d'uso (in inglese *Use Case Diagram*) costituiscono una tipologia di diagrammi [Unified Modeling Language \(UML\)](#) finalizzati alla rappresentazione delle funzionalità e dei servizi offerti da un sistema, dal punto di vista degli attori che interagiscono con esso.

3.2.1 Attori

Gli attori nei casi d'uso rappresentano entità esterne al sistema, quali utenti o altri servizi, che interagiscono con esso. In merito al sistema sviluppato si possono individuare come attori:

- **Sviluppatore:** è l'attore principale che utilizza il sistema, sia per generare e revisionare i test sia per consultare le dashboard di telemetria durante le attività di diagnosi e tuning.
- **LLM:** modello linguistico di grandi dimensioni ([LLM](#)) invocato dall'ambiente di sviluppo, utilizzato per la generazione automatica di test a partire dalle specifiche prodotte tramite *Spec-Kit* e per il supporto alla propagazione della telemetria ai moduli del framework.
- **Sia.Core:** applicazione *full-stack* oggetto dell'osservazione e del testing.

3.2.2 Elenco casi d'uso

- UC1 — Generazione automatica di test a partire da una specifica *Spec-Kit*.
- UC2 — Visualizzazione delle metriche, tracce e log di un modulo tramite dashboard Grafana.
- UC3 — Propagazione della telemetria a un nuovo modulo del framework tramite le *Claude skill* dedicate.
- UC4 — Diagnosi di una richiesta lenta a partire da un pannello percentili, tramite *exemplar* verso la tracce e navigazione ai log correlati.

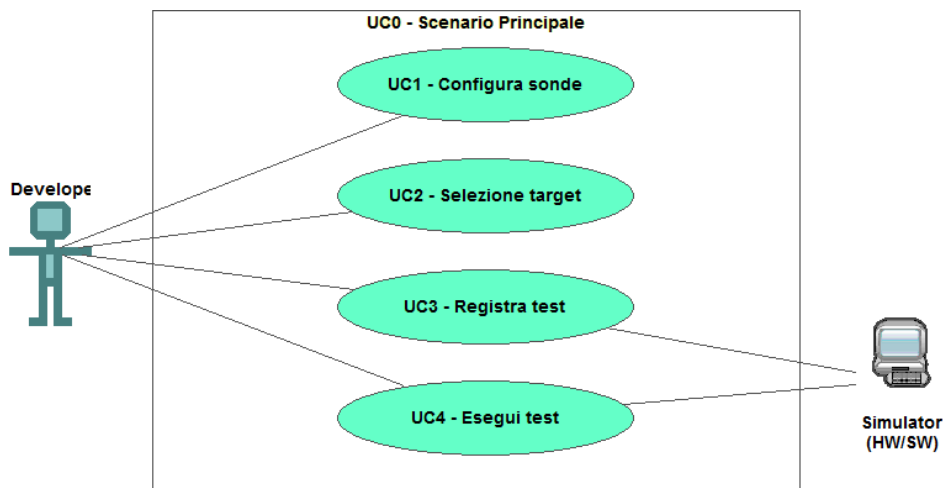


Figura 3.1: Use Case - UC0: Scenario principale

UC0: Scenario principale

Attori Principali: Sviluppatore applicativi.

Precondizioni: Lo sviluppatore è entrato nel plug-in di simulazione all'interno dell'IDE.

Descrizione: La finestra di simulazione mette a disposizione i comandi per configurare, registrare o eseguire un test.

Postcondizioni: Il sistema è pronto per permettere una nuova interazione.

3.3 Tracciamento dei requisiti

Da un'attenta analisi dei requisiti e degli use case effettuata sul progetto è stata stilata la tabella che traccia i requisiti in rapporto agli use case.

Sono stati individuati diversi tipi di requisiti e si è quindi fatto utilizzo di un codice identificativo per distinguerli.

Il codice dei requisiti è così strutturato $R(F/Q/V)(N/D/O)$ dove:

R = requisito

F = funzionale

Q = qualitativo

V = di vincolo

N = obbligatorio (necessario)

D = desiderabile

O = opzionale

Nelle tabelle 3.1, 3.2 e 3.3 sono riassunti i requisiti e il loro tracciamento con gli use case delineati in fase di analisi.

Tabella 3.1: Tabella del tracciamento dei requisiti funzionali

Requisito	Descrizione	Use Case
RFN-1	Tracciamento del percorso completo di ogni richiesta HTTP, incluse le operazioni sul database	UC2, UC4
RFN-2	Identificazione delle operazioni CRUD tramite attributi di metodo e repository	UC2, UC4
RFN-3	Esposizione di metriche operative per i moduli di autenticazione, accesso ai dati e griglia	UC2
RFN-4	Correlazione tra log e trace tramite <code>TraceId</code> e <code>SpanId</code>	UC4
RFN-5	Attivazione/disattivazione della telemetria tramite configurazione <i>appsettings</i>	UC3
RFN-6	Configurazione indipendente del campionamento per ciascuna sorgente di trace	UC3
RFN-7	Visualizzazione di trace e metriche raggruppate in dashboard Grafana	UC2
RFN-8	Raccolta dei dati da più servizi e installazioni, con filtraggio per cliente e servizio	UC2
RFN-9	Sviluppo di <i>Claude skill</i> per la propagazione della telemetria ai moduli del framework	UC3
RFN-10	Arricchimento automatico degli span con identità utente (<code>enduser.id</code> , <code>enduser.name</code>)	UC4
RFN-11	Esposizione di <i>exemplars</i> per le metriche Histogram (puntatore al <code>TraceId</code>)	UC4

Tabella 3.2: Tabella del tracciamento dei requisiti qualitativi

Requisito	Descrizione	Use Case
RQN-1	La telemetria non deve degradare sensibilmente le prestazioni in produzione	-
RQN-2	Le metriche non devono generare un numero eccessivo di serie temporali in Prometheus (cardinalità controllata)	-
RQD-1	Le dashboard devono privilegiare la visualizzazione della latenza (percentili) rispetto al solo <i>throughput</i>	UC2

Tabella 3.3: Tabella del tracciamento dei requisiti di vincolo

Requisito	Descrizione	Use Case
RVN-1	I segnali prodotti devono essere compatibili con qualsiasi backend OTel-compatibile	-
RVN-2	Le impostazioni devono essere modificabili per ambiente (sviluppo, <i>staging</i> , produzione) tramite <i>appsettings</i>	-

Capitolo 4

Tecnologie adottate

Il presente capitolo descrive le tecnologie adottate per lo sviluppo del prodotto e ne illustra le principali caratteristiche.

4.1 Studio e scelta delle tecnologie

Una parte fondamentale dello stage è stata dedicata all'attività di ricerca, analisi e selezione delle tecnologie più adeguate per lo sviluppo del progetto. Questa fase iniziale ha rivestito un ruolo cruciale, in quanto le scelte tecnologiche influenzano in modo significativo sia la qualità del prodotto finale sia la sostenibilità dello sviluppo nel lungo periodo. Il processo di selezione è iniziato con una prima fase di scrematura delle soluzioni disponibili sul mercato, effettuata sulla base dei requisiti e delle esigenze espresse dall'azienda. In particolare, sono stati considerati diversi criteri di valutazione, tra cui le funzionalità offerte dalle tecnologie, i costi associati al loro utilizzo e il grado di integrazione con il sistema già esistente. Successivamente, le tecnologie ritenute più promettenti sono state sottoposte a una fase di sperimentazione pratica attraverso lo sviluppo di [Proof of Concept \(PoC\)](#). Questa attività ha permesso di valutare concretamente le prestazioni, l'efficacia e la compatibilità delle soluzioni selezionate, fornendo elementi utili per una scelta consapevole e motivata delle tecnologie da adottare nel progetto.

4.2 Tecnologie e strumenti

Di seguito è fornita una descrizione generale delle tecnologie e degli strumenti utilizzati per lo sviluppo del progetto.

Angular

Framework open-source per lo sviluppo di applicazioni *web front-end*, basato su *TypeScript* e mantenuto da *Google*.

- **Architettura a componenti:** *Angular* promuove una struttura modulare basata su componenti riutilizzabili, facilitando la manutenzione e la scalabilità dell'applicazione.

- **Strumenti integrati:** offre soluzioni native per *routing*, gestione dello stato, form e comunicazione con [API](#), riducendo la necessità di librerie esterne.
- **Supporto alle SPA:** è progettato per lo sviluppo di [Single Page Application \(SPA\)](#), garantendo un'esperienza utente fluida e dinamica grazie all'aggiornamento asincrono dei contenuti.



Figura 4.1: Logo Angular

.NET

Piattaforma di sviluppo *software open-source* e multiplatforma sviluppata da *Microsoft*, utilizzata per la creazione di applicazioni web, desktop, mobile e cloud.

- **Elevate prestazioni e scalabilità:** .NET garantisce alte performance e una gestione efficiente delle risorse, risultando adatto ad applicazioni *enterprise* e ad alto carico.
- **Architettura strutturata e flessibile:** supporta *pattern* architetturali come *Clean Architecture* e *Dependency Injection*, favorendo la manutenibilità e l'organizzazione del codice.
- **Ampio ecosistema e integrazione:** mette a disposizione numerose librerie e strumenti per lo sviluppo di [API](#), gestione dei dati, sicurezza e integrazione con servizi esterni.



Figura 4.2: Logo .NET

Spec-Kit

Strumento utilizzato per definire in modo strutturato e formale le specifiche da fornire a modelli di intelligenza artificiale per la generazione automatica di codice. Consente di rendere il processo *AI-driven* più tracciabile, riproducibile e controllabile, migliorando la qualità dell'output generato, in particolare nel contesto della generazione di test software.

**Figura 4.3:** Logo Spec-Kit

OpenTelemetry

Insieme di strumenti, [API](#) e librerie *open-source* per la raccolta, generazione ed esportazione di dati di telemetria, quali metriche, *log* e *trace*, provenienti da applicazioni software. È utilizzato per implementare sistemi di osservabilità distribuita, permettendo il monitoraggio del comportamento e delle prestazioni di un sistema in tempo reale.

**Figura 4.4:** Logo OpenTelemetry

Grafana

Piattaforma *open-source* per la visualizzazione e l'analisi di dati, utilizzata principalmente per il monitoraggio di sistemi e applicazioni. Consente di creare dashboard interattive a partire da diverse fonti di dati (come *OpenTelemetry*), facilitando l'interpretazione delle metriche e l'individuazione di anomalie o criticità. Nel progetto è stato adottato lo stack *LGTM* (Loki per i *log*, Grafana per la visualizzazione, Tempo per le *trace*, Prometheus/Mimir per le *metriche*).

**Figura 4.5:** Logo Grafana

Serilog

Libreria di *logging* strutturato per .NET, scelta per la produzione e la gestione dei *log* applicativi. Supporta nativamente il *sink* OTLP, permettendo l'esportazione dei *log* verso il *Collector* OpenTelemetry e la correlazione automatica con le *trace* tramite *TraceId* e *SpanId* iniettati da *Activity.Current*.

Playwright

Framework *open-source* per l'automazione dei browser e il testing end-to-end di applicazioni web. Permette di simulare il comportamento reale degli utenti attra-

verso interazioni come navigazione, compilazione di form e verifica dei contenuti, supportando diversi browser (Chromium, Firefox, WebKit).



Figura 4.6: Logo Playwright

Vitest

Framework di testing unitario moderno per applicazioni JavaScript e TypeScript, progettato per integrarsi con strumenti di sviluppo basati su Vite. Offre esecuzione rapida dei test, e funzionalità avanzate come mocking, snapshot testing e coverage del codice.



Figura 4.7: Logo Vitest

Claude Code

Claude Code è l'agente di codifica basato su modelli linguistici di grandi dimensioni (LLM) utilizzato durante lo stage come *pair engineer* strutturato per lo studio delle tecnologie, la realizzazione di prototipi e la propagazione della telemetria ai moduli del framework. Il MCP è il protocollo che consente all'agente di interagire con servizi esterni (in particolare *Grafana*) in modo strutturato, ottenendo dati runtime e invocando operazioni come la creazione e la validazione di dashboard. L'utilizzo combinato di *Claude Code* e MCP è descritto in dettaglio nel capitolo 6.

Capitolo 5

Progettazione

In questo capitolo si esporranno le caratteristiche tecniche della soluzione prodotta, descrivendo la progettazione e l'architettura adottata

5.1 Osservabilità nei sistemi distribuiti

L'osservabilità è la proprietà di un sistema software che permette di comprenderne lo stato interno esaminando esclusivamente i dati prodotti in output. Nel contesto delle applicazioni web distribuite, questa proprietà è resa possibile dalla produzione sistematica di tre categorie di segnali:

- **Logs:** rappresentano eventi discreti generati dal sistema durante la sua esecuzione. Contengono informazioni dettagliate su operazioni svolte, errori, stati e messaggi diagnostici, risultando fondamentali per attività di debugging e analisi puntuale di anomalie.
- **Traces:** descrivono il flusso di esecuzione di una richiesta all'interno del sistema, tracciandone il percorso attraverso i vari componenti e servizi. Sono particolarmente utili in architetture distribuite, in quanto permettono di analizzare le dipendenze tra servizi e individuare eventuali colli di bottiglia o punti di errore.
- **Metriche:** rappresentano misurazioni numeriche aggregate nel tempo, utilizzate per monitorare lo stato e le prestazioni del sistema. Si distinguono in:
 - *Metriche di performance:* raccolte automaticamente tramite strumenti come OpenTelemetry, includono informazioni quali tempi di risposta, utilizzo delle risorse, throughput e latenza, utili per valutare le prestazioni del sistema.
 - *Metriche di business:* definite a livello applicativo, riguardano aspetti funzionali del dominio, come numero di operazioni effettuate, utenti attivi o eventi specifici, e consentono di monitorare l'utilizzo e il valore del sistema dal punto di vista aziendale.

La forza di un sistema di osservabilità moderno non risiede nell'utilizzo isolato di questi segnali, bensì nella loro correlazione: poter passare da una metrica anomala alla trace corrispondente, e da questa ai log emessi durante quella specifica richiesta, costituisce uno strumento diagnostico di grande efficacia.

5.2 Lo standard OpenTelemetry

OTel è un framework open-source, oggi standard de facto nel settore, che fornisce un modello unificato e vendor-neutral per la produzione, la raccolta e l'esportazione di segnali di osservabilità. È il risultato della fusione tra i progetti *OpenCensus* e *OpenTracing*, avvenuta nel 2019 sotto l'egida della Cloud Native Computing Foundation (CNCF).

I principi fondamentali di **OTel** rilevanti per questo progetto sono:

- **Separazione tra produzione ed esportazione.** Le **API** applicative sono indipendenti dal backend di storage scelto: lo stesso codice può inviare dati a Grafana, Jaeger, Zipkin o qualsiasi altro sistema compatibile.
- **Instrumentazione automatica.** Per i framework più diffusi (ASP.NET Core, HttpClient, SqlConnection), **OTel** fornisce librerie che non richiedono modifiche al codice applicativo.
- **Estensibilità.** È possibile aggiungere attributi custom, definire metriche applicative proprie e configurare sampler e processor personalizzati.

5.3 Architettura della telemetria

Il sistema di telemetria sviluppato si basa su una distinzione tra due macro-componenti principali: la **produzione dei dati** lato applicazione e la **raccolta e visualizzazione** lato infrastruttura.

L'applicazione backend, sviluppata in ambiente *.NET*, utilizza lo standard open-source OpenTelemetry per generare i tre principali segnali di osservabilità: *trace*, *metriche* e *log*. Questi dati vengono successivamente inviati a un'infrastruttura centralizzata basata sullo stack Grafana, che ne consente l'analisi e la visualizzazione.

5.3.1 Produzione dei dati di telemetria

La generazione dei dati avviene direttamente all'interno dell'applicazione backend tramite diverse tecnologie integrate.

Le **trace** vengono prodotte utilizzando OpenTelemetry, che si appoggia alle API native di .NET (*ActivitySource* e *Activity*) per rappresentare le operazioni eseguite dal sistema sotto forma di *span*. Ogni richiesta viene quindi tracciata lungo tutto il suo ciclo di vita, includendo chiamate HTTP, operazioni sui database e logica applicativa.

Le **metriche** vengono generate attraverso due approcci distinti:

- metriche di *performance*, raccolte automaticamente da OpenTelemetry (ad esempio durata delle richieste HTTP o utilizzo delle risorse);

- metriche di *business*, definite manualmente tramite le API **Meter** di .NET, per monitorare aspetti specifici del dominio applicativo.

I **log** vengono invece gestiti tramite la libreria Serilog, che consente la produzione di log strutturati e la loro esportazione verso l'infrastruttura di osservabilità. Grazie all'integrazione con OpenTelemetry, i log risultano automaticamente correlati alle trace tramite identificativi condivisi (*TraceId* e *SpanId*).

5.3.2 Integrazione con OpenTelemetry Collector ed ecosistema Grafana

Per la gestione e la visualizzazione dei dati di telemetria è stata adottata un'architettura basata su OpenTelemetry Collector e sullo stack Grafana (LGTM: Loki, Grafana, Tempo e Prometheus). I dati di telemetria generati dall'applicazione backend, vengono inizialmente prodotti tramite le librerie OpenTelemetry, .NET Meters e Serilog, e poi vengono esportati in formato compatibile OpenTelemetry. Tutti questi dati vengono inviati a un componente intermedio, l'*OpenTelemetry Collector*, tramite protocollo OTLP (OpenTelemetry Protocol). Il Collector svolge un ruolo centrale nell'architettura di osservabilità, fungendo da punto di raccolta e smistamento dei dati. Le sue principali responsabilità includono:

- **Ricezione dei dati:** acquisizione di trace, metriche e log dall'applicazione tramite endpoint OTLP;
- **Elaborazione:** possibilità di applicare trasformazioni, filtri o arricchimenti ai dati raccolti;
- **Esportazione:** inoltro dei dati verso i sistemi di destinazione tramite specifici *exporter*.

Nel progetto, il Collector è configurato per instradare i diversi tipi di dati verso componenti distinti dello stack Grafana:

- le **trace** vengono esportate verso Tempo, permettendo l'analisi dei flussi di esecuzione e delle dipendenze tra servizi;
- le **metriche** vengono inviate a Prometheus, rendendo possibile il monitoraggio delle prestazioni e la creazione di dashboard;
- i **log** vengono esportati verso Loki, consentendo la consultazione centralizzata e la correlazione con trace e metriche .

Grafana rappresenta il punto di accesso unificato a queste informazioni, offrendo strumenti per la visualizzazione e l'analisi dei dati di telemetria. In particolare, permette di costruire dashboard personalizzate basate su metriche, esplorare i log e navigare le trace, abilitando una visione completa e correlata del comportamento del sistema. Questa architettura consente di separare la raccolta dei dati dalla loro visualizzazione, migliorando la scalabilità e la flessibilità del sistema di monitoraggio, oltre a facilitare eventuali estensioni future.

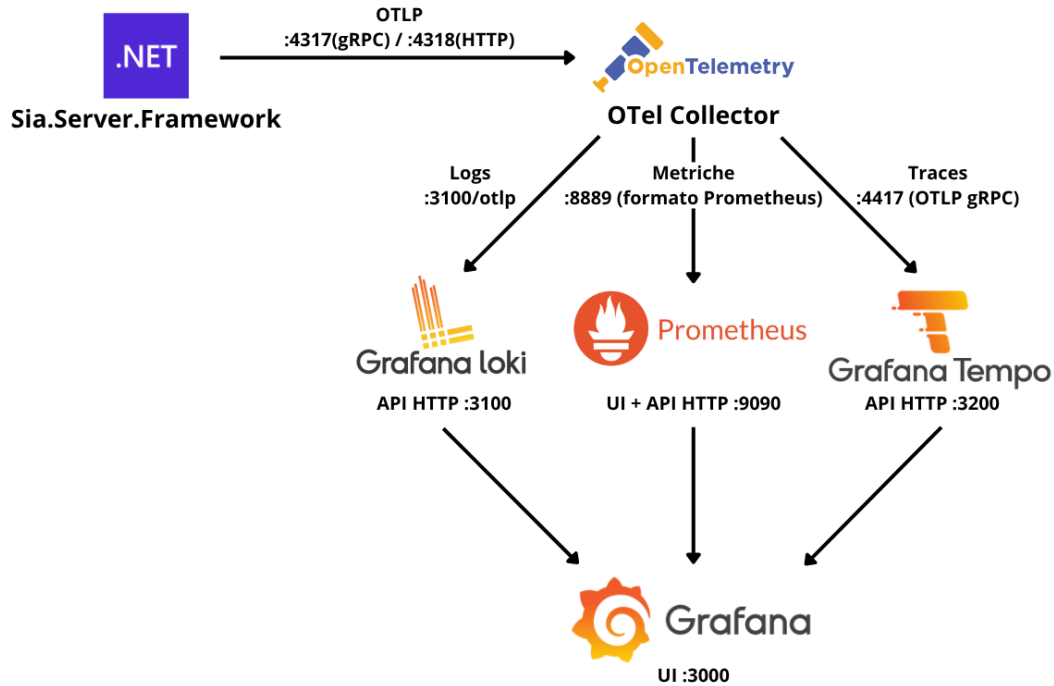


Figura 5.1: Architettura OTEL Collector

5.3.3 Scelte architetturali

5.3.3.1 Adozione di OpenTelemetry

La scelta di **OTel**, come richiesto dal requisito RVN-1, permette flessibilità nella scelta del backend di visualizzazione. Le alternative considerate, soluzioni proprietarie come Azure Application Insights o Datadog APM, avrebbero introdotto un accoppiamento stretto con un singolo fornitore, limitando la flessibilità nella scelta del backend e rendendo difficoltosa la migrazione futura.

5.3.3.2 Architettura opt-in

L'intero sistema è progettato come *opt-in*: la telemetria è attiva solo se `OpenTelemetry.Enabled = true` in `appsettings`, rispettando così il requisito RFN-5. Questo consente di abilitarla esclusivamente negli ambienti che la richiedono, evitando *overhead* in sviluppo locale.

5.3.3.3 Centralizzazione della configurazione

Tutta la configurazione **OTel** è centralizzata in `BaseApplicationBuilder`, componente condiviso da tutti i prodotti *Sia.Web*. I singoli moduli si limitano a registrare il proprio `ActivitySource` e `Meter` nel provider già costruito, senza conoscere i dettagli del pipeline.

5.3.3.4 Campionamento

Il campionamento è gestito da un sampler custom, che combina una logica *ratio-based* per le *root span* con ereditarietà della decisione per le *child span*.

Se lo span ha un parent valido (`TraceId` e `SpanId` non nulli), il sampler eredita la decisione del parent senza ricalcolo. Questo garantisce la coerenza interna della trace: se la root viene campionata, tutte le child span, vengono raccolte integralmente.

Per le root span, il sampler risolve il ratio da applicare tramite `ResolveRatio`: il nome dell'activity (es. `sia.grid.page`) viene confrontato con i prefissi derivati dai nomi source configurati in `appsettings` (es. `Sia.Grid` → `sia.grid.`). I prefissi sono ordinati per lunghezza decrescente, così il match più specifico ha sempre precedenza su quello più generale. Se nessun prefisso corrisponde, si applica il ratio globale `SamplingRatio`.

5.3.3.5 Arricchimento automatico del contesto utente

Per soddisfare il requisito RFN-10, il sistema arricchisce automaticamente ogni span con l'identità dell'utente autenticato senza richiedere modifiche ai singoli moduli applicativi. Il meccanismo è descritto nel dettaglio nella sezione ??.

5.3.3.6 Correlazione metrica-trace tramite *exemplars*

Per soddisfare il requisito RFN-11, le metriche di tipo *Histogram* espongono *exemplars*, ovvero campioni che portano con sé il `TraceId` della richiesta che li ha generati. Questo abilita la navigazione dal pannello di una metrica aggregata alla trace specifica che ha prodotto il campione anomalo, risolvendo la domanda “quale richiesta ha causato questo picco?”. Il meccanismo è descritto nel dettaglio nella sezione ??.

5.3.3.7 Pattern architetturali applicati

La soluzione si appoggia a due *design pattern* classici per risolvere problemi ricorrenti: il pattern *Adapter*, utilizzato per riconciliare i `Counter` per-modulo con un contratto comune (`IControllerDiagnostics`), e il pattern *Factory*, utilizzato per costruire gli oggetti del pipeline `OTel` a partire dalla configurazione `appsettings`. I dettagli sono illustrati nella sezione 5.5.

5.4 Modello dei segnali e utilizzo degli attributi

5.4.1 Modello dei log

I log sono prodotti da Serilog con l'interfaccia `ILogger<T>` e arricchiti automaticamente con `TraceId` e `SpanId` quando emessi nel contesto di una richiesta tracciata. Questo abilita la navigazione bidirezionale tra log e trace in Grafana.

5.4.2 Modello delle trace

Una *trace* è l'insieme completo di un'operazione end-to-end, identificata da un unico `trace_id`. È composta da una gerarchia di *span*, ciascuno con un proprio `span_id` e, ad eccezione della root, un `parent_span_id`.

Nel sistema, le trace sono prodotte da tre tipologie di sorgenti: strumentazione automatica HTTP (richieste in ingresso via `AspNetCore`, chiamate in uscita via `HttpClient`); strumentazione automatica database (query SQL Server via `SqlClient`, query PostgreSQL via `Npgsql`); strumentazione manuale dei moduli di `Sia.Core` tramite le classi `Sia[NomeModulo]Diagnostics`.

5.4.3 Modello delle metriche

Sono impiegati quattro tipi di strumento metrico, offerti dall'`API System.Diagnostics.Metrics` di `.NET`:

Counter Contatore monotono crescente per eventi cumulativi (es. login riusciti, query eseguite). In Prometheus acquisisce il suffisso `_total`.

Histogram Distribuzione statistica per latenze e percentili (p50, p95, p99). In Prometheus genera tre serie: `_bucket`, `_sum`, `_count`.

UpDownCounter Contatore non monotono, che può essere incrementato o decrementato, utilizzato per rappresentare grandezze “in corso” (es. numero di operazioni attualmente in esecuzione).

ObservableGauge Strumento campionato tramite *callback* periodica, utilizzato per valori istantanei non cumulativi (es. numero di connessioni attive).

La progettazione delle metriche ha seguito due criteri guida. Il primo è la *bassa cardinalità*: i tag con alta variabilità (es. identificatori numerici di record) sono applicati solo agli span, mai alle metriche, per evitare un'esplosione del numero di serie temporali in Prometheus. Il secondo è la *pre-aggregazione*: le metriche vengono aggregate lato applicazione prima dell'esportazione, con overhead nettamente inferiore rispetto alla raccolta di trace complete.

5.4.4 Tassonomia degli attributi

5.4.4.1 Resource attributes (comuni a tutti i segnali)

Attributi presenti in tutte le metriche, tracce e log di tutti i moduli.

Tabella 5.1: Resource attributes comuni a tracce, metriche e log

Attributo	Chiave appsettings	Scopo
<code>service.name</code>	<code>ServiceName</code>	Identifica l'installazione
<code>sia.product_type</code>	<code>ProductType</code>	Linea di prodotto
<code>sia.customer_name</code>	<code>CustomerName</code>	Nome del cliente

5.4.4.2 Attributi del modulo Sia.Base

Tutti i tag utilizzati nel modulo Sia.Base, modulo che si occupa delle chiamate CRUD al database, sono i medesimi sia per le tracce che per le metriche.

Tabella 5.2: Tag del modulo Sia.Base (operazioni [CRUD](#))

Tag	Significato	Valori esempio
<code>sia.operation</code>	Tipo di operazione CRUD	<code>insert</code> , <code>update</code> , <code>delete</code> , <code>find_one</code> , <code>find_many</code> , <code>relaz_check</code> , <code>custom</code>
<code>sia.method</code>	Metodo infrastrutturale	<code>InsertSiaEntity</code> , <code>UpdateSiaEntity</code> , <code>FindSiaEntities</code>
<code>sia.caller_method</code>	Metodo business chiamante	<code>CreateOrder</code> , <code>SaveInvoice</code>
<code>sia.entity</code>	Tipo C# dell'entità	<code>OrderEntity</code> , <code>UserEntity</code>
<code>sia.repository</code>	Classe repository	<code>OrdersRepository</code> , <code>CatalogRepository</code>
<code>sia.relaz.kind</code>	Tipo di controllo relazione	<code>relaz</code> , <code>view-relaz</code>
<code>exception.type</code>	Tipo eccezione (solo errori)	<code>SqlException</code> , <code>TimeoutException</code>

5.4.4.3 Attributi del modulo Sia.Auth

I tag nelle metriche del modulo Sia.Auth sono a bassa cardinalità, di fatto solo la metrica `sia.auth.login.failure` contiene il tag `reason` che descrive il motivo dell'errore.

Mentre le tracce, contengono i seguenti attributi:

Tabella 5.3: Tag degli span del modulo Sia.Auth

Tag	Valori possibili
auth.method	credentials
auth.result	success, unauthorized, error, bad_request, two_factor_required
auth.two_factor_required	true (solo se 2FA richiesto)
auth.two_factor_type	email, totp

5.4.4.4 Attributi del modulo Sia.Grid

Le metriche, come le trace del modulo Sia.Grid, contengono tutte l'attributo `componentId`.

Tabella 5.4: Tag degli span del modulo Sia.Grid

Span	Tag
sia.grid.page	componentId, hasSpName
sia.grid.summary	componentId
sia.grid.massive_delete	componentId, spName

5.4.5 Tracciamento dell'identità utente

Ogni trace generata da una chiamata autenticata riporta automaticamente l'identità dell'utente tramite i tag `enduser.id` ed `enduser.name`, senza che i singoli moduli debbano iniettarli manualmente (RFN-10).

Il meccanismo si articola in due componenti trasparenti al codice applicativo:

un middleware ASP.NET Core (`EnrichActivityWithUserMiddleware`), registrato dopo `UseAuthentication()`, legge gli attributi `userId` e `userName` da `HttpContext.User` e le pubblica nel Baggage [OTel](#).

un ActivityProcessor custom (`BaggageActivityProcessor`) intercetta l'hook `OnStart` di ogni nuova `Activity` e copia le entry con prefisso `enduser.` come tag sullo span. Poiché il baggage viene ereditato dalle child span, tutte le query SQL generate nell'ambito della stessa richiesta portano gli stessi `enduser.*` della root.

Gli attributi `enduser.*` non vengono mai aggiunti come tag delle metriche: farlo farebbe crescere il numero di serie temporali di Prometheus linearmente con il numero di utenti, violando RQN-2. La correlazione `metrica → utente` si ottiene invece tramite gli exemplar: dalla metrica si naviga alla trace, che porta già `enduser.*`.

5.4.6 Exemplar: collegamento diretto da metrica a trace

Le metriche aggregate dicono *quanto* e *quando* qualcosa è andato storto, ma non *quale* richiesta specifica ne è responsabile. Un picco sul percentile p_{99} della latenza,

ad esempio, non rivela di per sé la chiamata incriminata. Gli *exemplar* colmano questo divario: sono campioni concreti allegati ai dati aggregati di una metrica che trasportano il `TraceId` della richiesta che li ha prodotti. In Grafana, si materializzano come piccoli diamanti sovrapposti ai grafici istogramma; un click su uno di essi apre direttamente la trace corrispondente in Tempo, dalla quale si può proseguire verso i log correlati. Il risultato è una catena di navigazione completa (metrica → trace → utente → log).

L'utilizzo degli *exemplar* è possibile grazie ad una feature di *Prometheus* chiamata *exemplar-storage*. Quest'ultima è stata affiancata al filtro `TraceBased` che risolve il problema dei puntatori orfani, ovvero quei puntatori che puntano ad una trace scartata dal sampling. Questa fusione garantisce dunque che un *exemplar* venga prodotto soltanto quando la trace a cui punta è effettivamente disponibile, azzerando l'overhead sulle richieste non campionate.

5.5 Design pattern applicati

Oltre ai pattern architetturali generali citati nella sezione 5.3.3.7, la soluzione utilizza due *design pattern* della letteratura classica (Gamma et al.¹) per risolvere problemi ricorrenti dell'strumentazione.

5.5.1 Standardizzazione del counter errori controller tramite pattern Adapter

Contesto

In un'architettura a moduli come quella di *Sia.Web*, ogni modulo possiede una propria classe di diagnostica statica con i propri `Counter` e `Histogram`, registrati sul `Meter` specifico del modulo. Questo garantisce isolamento e flessibilità, ma introduce un rischio di divergenza: la logica di registrazione degli errori nei controller, che richiede l'unwrap di `SiaLoggerException.OriginalException` e l'applicazione dei tag `exception.type` e `sia.operation`, rischia di essere replicata in modo leggermente diverso in ciascun controller, rendendo le metriche non omogenee e difficili da confrontare in Grafana.

Problema

Il nodo architetturale da risolvere è che le classi di diagnostica dei moduli sono `static` e non possono implementare direttamente un'interfaccia di istanza. Il problema viene risolto con il pattern **Adapter**: per ciascun modulo si definisce una classe annidata privata che adatta il counter statico al contratto comune, e la classe di diagnostica espone un singleton `AsControllerDiagnostics` di tipo `IControllerDiagnostics`. I field statici esistenti restano invariati, garantendo piena retrocompatibilità con i chiamanti diretti.

Soluzione

La soluzione adottata promuove questa logica a comportamento standard del framework: `BaseApiController` espone un metodo protetto `RecordMetricError` che

¹Erich Gamma et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

ogni controller derivato può invocare passando la propria classe di diagnostica. Il metodo accetta un parametro di tipo `IControllerDiagnostics`, un'interfaccia che impone la presenza di un unico membro: il `Counter<long> ControllerErrors`.

Listing 5.1: Pattern Adapter applicato alla diagnostica del modulo CRUD

```
public static class SiaDevextremeCrudDiagnostics
{
    // field statico esistente: invariato e retrocompatibile
    public static readonly Counter<long> ControllerErrors =
        ...;

    // singleton che adatta il counter al contratto comune
    public static readonly IControllerDiagnostics
        AsControllerDiagnostics
        = new ControllerDiagnosticsAdapter();

    private sealed class ControllerDiagnosticsAdapter :
        IControllerDiagnostics
    {
        public Counter<long> ControllerErrors
            => SiaDevextremeCrudDiagnostics.ControllerErrors;
    }
}
```

Nel controller, la chiamata diventa:

Listing 5.2: Invocazione di `RecordMetricError` dal controller derivato

```
RecordMetricError(SiaDevextremeCrudDiagnostics.
    AsControllerDiagnostics, ex, operation);
```

In questo modo abbiamo ottenuto una registrazione uniforme da parte dei vari controller, tramite contratto (l'interfaccia) applicata dalla classe base (il metodo), eliminando duplicazione e divergenze di naming tra moduli.

5.5.2 Pattern Adapter

Contesto:

Il framework fornisce un metodo condiviso `BaseApiController.RecordMetricError(IControllerDiagnosticsException, string)` che incapsula la logica di *unwrap* di `SiaLoggerException.OriginalException` e l'applicazione dei *tag exception.type* e *sia.operation* su una metrica `Counter`. Ogni modulo con controller deve riusare tale metodo, ma ciascun modulo ha il proprio `Counter<long> ControllerErrors` registrato sul proprio `Meter` con naming `sia.{modulo}.controller.errors`.

Problema:

Il metodo condiviso dichiara un parametro del tipo astratto `IControllerDiagnostics`. I moduli però espongono un contatore concreto come membro statico di una classe `Sia{Modulo}Diagnostics`. Occorre riconciliare il contratto astratto con la pluralità di implementazioni concrete, senza duplicare la logica di *unwrap* in ogni controller.

Soluzione:

Per ciascun modulo viene definita una classe annidata `private sealed class ControllerDiagnosticsAdapter : IControllerDiagnostics`, che adatta il *counter* statico del modulo al contratto comune. La classe `Sia{Modulo}Diagnostics` espone poi un singleton `public static readonly IControllerDiagnostics AsControllerDiagnostics`. Il controller chiama sempre:

```
RecordMetricError(Sia{Modulo}Diagnostics.AsControllerDiagnostics, ex, nameof(Metodo))
```

senza conoscere l'implementazione concreta.

5.5.2.0.1 Vantaggi.

- contratto unico a livello framework, *counter* distinti per modulo (cardinalità controllata);
- logica di *unwrap* e applicazione *tag* scritta una sola volta;
- zero ripetizione nel controller: ogni `catch` diventa una singola riga.

Riferimento canonico: `SiaDevextremeCrudDiagnostics.AsControllerDiagnostics` nel modulo `Sia.Devextreme.Crud`.

5.5.3 Pattern Factory

Il pattern *Factory* è applicato in più punti del pipeline [OTel](#) per mantenere in un unico luogo la traduzione da configurazione *appsettings* a oggetti del runtime OpenTelemetry.

5.5.3.0.1 Costruzione del *sampler*. La creazione di `SiaParentBasedSampler` avviene in un metodo *factory* che, letti il valore globale `SamplingRatio` e i *ratio* per-*source* da *appsettings*, restituisce l'istanza del *sampler* configurata correttamente. L'invocazione avviene una sola volta in `BaseApplicationBuilder`; il resto del pipeline consuma un oggetto già pronto.

5.5.3.0.2 Registrazione di *Meter* e *ActivitySource*. Le classi `Sia{Modulo}Diagnostics` agiscono come *static factory* centralizzata per gli strumenti di osservabilità del modulo: istanziano al *type loading* il `Meter`, gli `ActivitySource`, e i `Counter/Histogram` esposti come membri statici pubblici. Il codice applicativo non istanzia mai direttamente uno strumento OTel, ma accede sempre all'istanza fornita dalla classe `Diagnostics` del proprio modulo.

5.5.3.0.3 Costruzione del *ResourceBuilder*. Gli attributi di *resource* (`service.name`, `sia.product_type`, `sia.customer_name`) vengono letti da *appsettings* e applicati a un unico `Resource` condiviso da *trace*, metriche e *log*, garantendone la coerenza cross-segnale. Anche questa costruzione avviene in un metodo *factory* interno a `BaseApplicationBuilder`.

5.5.3.0.4 Vantaggi.

- la configurazione da *appsettings* viene tradotta in oggetti OTel in un solo punto, semplificando i test e le variazioni per ambiente;
- i moduli vedono solo istanze pronte all'uso, senza conoscere il pipeline;
- è possibile sostituire la *factory* con una variante *no-op* per i test senza toccare il codice di business.

5.6 Diagrammi delle classi del sistema di telemetria

Questa sezione riassume, con tre diagrammi delle classi, la struttura complessiva del sistema di telemetria descritto nei paragrafi precedenti. I diagrammi sono in stile [UML](#) semplificato: rettangoli con nome e membri salienti, frecce tratteggiate per le dipendenze di uso, freccia con triangolo vuoto per la relazione di implementazione di interfaccia.

5.6.1 Pipeline condiviso in BaseApplicationBuilder

La figura 5.2 mostra il pipeline OTel centralizzato: i tre provider `TracerProvider`, `MeterProvider` e `LoggerProvider` sono costruiti una sola volta da `BaseApplicationBuilder`, che vi registra rispettivamente il `SiaParentBasedSampler`, il `PerSourceFilterProcessor` e il `BaggageActivityProcessor` per le *trace*, il filtro *exemplar* `TraceBased` per le metriche, e il *sink* OTLP di Serilog per i *log*. Tutti i segnali condividono lo stesso `ResourceBuilder`, garantendo la coerenza degli attributi `service.name`, `sia.product_type`, `sia.customer_name` e l'esportazione verso il *Collector* tramite `OtlpExporter`.

5.6.2 Pattern Sia{Modulo}Diagnostics e Adapter

La figura 5.3 illustra in forma generica la classe `Sia{Modulo}Diagnostics` che ogni modulo del framework espone. La classe è l'unico punto di istanziazione degli strumenti OTel del modulo (pattern *Static Factory*); inoltre, per i moduli che espongono un controller, ospita un `ControllerDiagnosticsAdapter` annidato che implementa il contratto condiviso `IControllerDiagnostics` (pattern *Adapter*). Il controller chiama il metodo condiviso `BaseApiController.RecordMetricError` passando il singleton `AsControllerDiagnostics`, senza conoscere l'implementazione concreta.

5.6.3 Arricchimento del contesto utente

La figura 5.4 mostra le collaborazioni che realizzano l'arricchimento automatico di ogni `Activity` con i *tag* `enduser.id` e `enduser.name`, descritte nella sezione ???. Il `EnrichActivityWithUserMiddleware` legge le *claim* da `HttpContext.User` e pubblica le voci nel `Baggage`; il `BaggageActivityProcessor`, registrato come `ActivityProcessor` nel `TracerProvider`, copia ogni *entry* di *baggage* con prefisso `enduser.` come *tag* sull'`Activity` nel proprio *hook* `OnStart`. L'effetto è che qualsiasi `Activity` creata

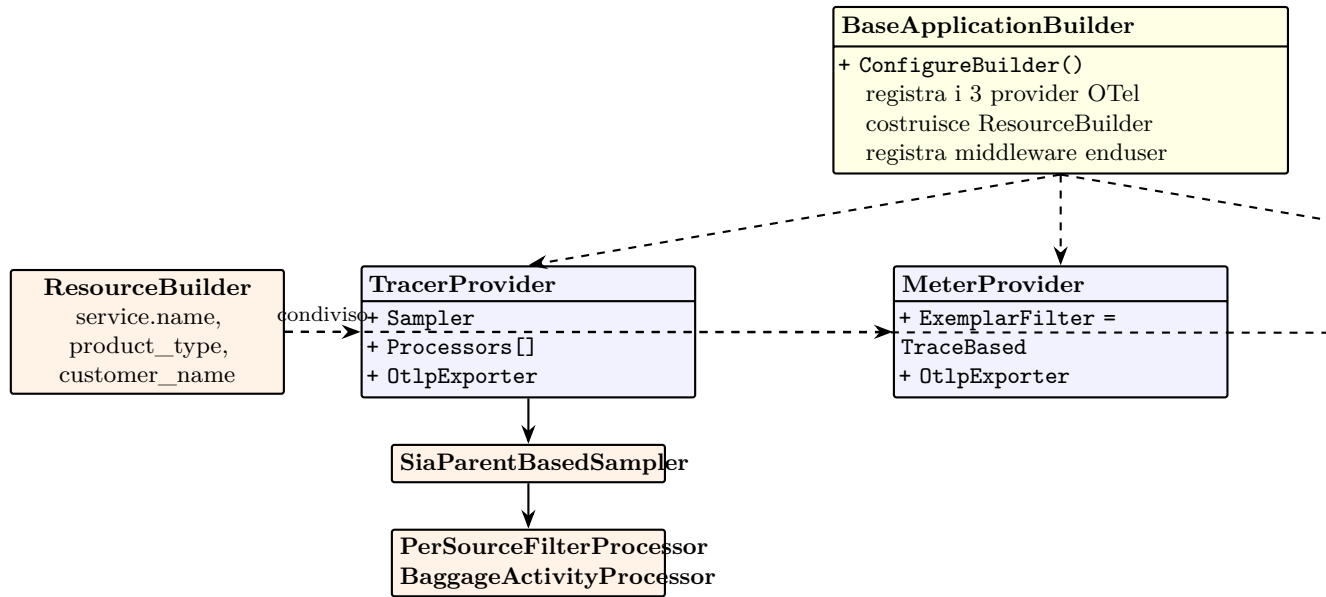


Figura 5.2: Pipeline OpenTelemetry centralizzato in `BaseApplicationBuilder`: i tre provider (trace, metriche, log) condividono lo stesso `ResourceBuilder` e utilizzano *sampler* e *processor* custom per le *trace*.

durante la richiesta — sia dalle classi `Sia{Modulo}Diagnostics` che dalle instrumentazioni automatiche di ASP.NET Core e `SqlClient` — riceve i *tag* utente senza modifiche al codice del modulo.

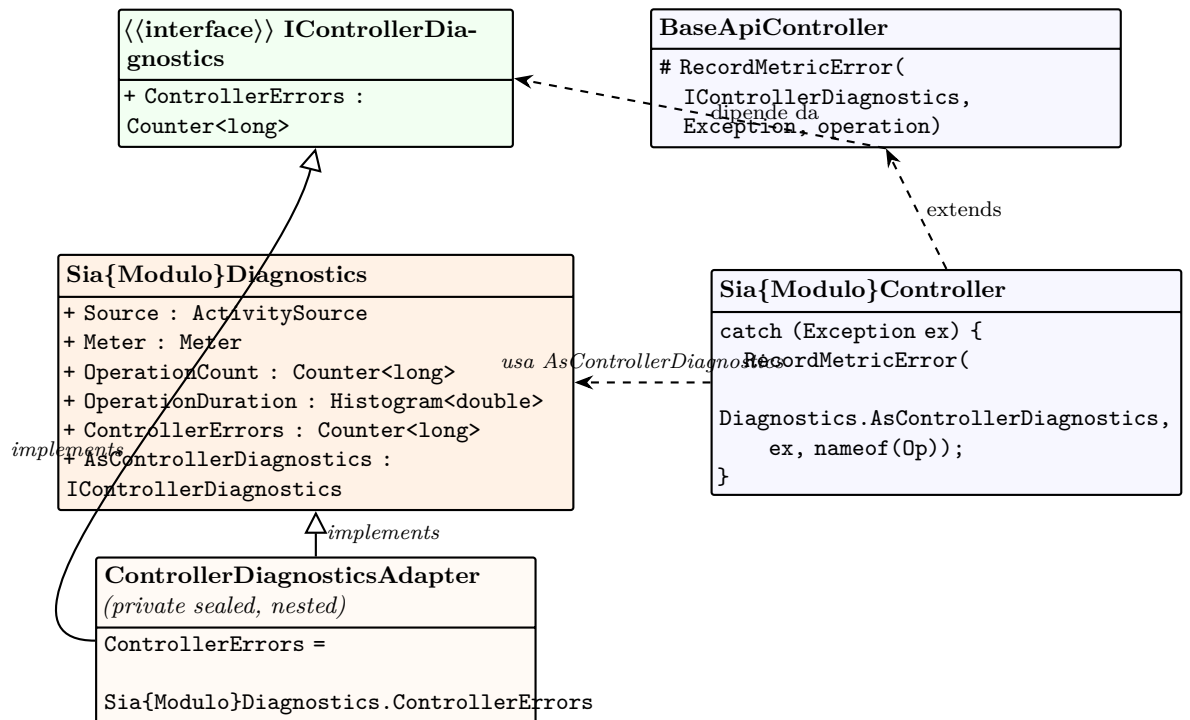


Figura 5.3: Struttura generica del pattern `Sia{Modulo}Diagnostics` con `ControllerDiagnosticsAdapter` annidato. Il controller invoca `RecordMetricError` del metodo condiviso `BaseApiController`, passando il singleton `AsControllerDiagnostics`; l'Adapter riconcilia il `Counter` per-modulo con il contratto comune `IControllerDiagnostics`.

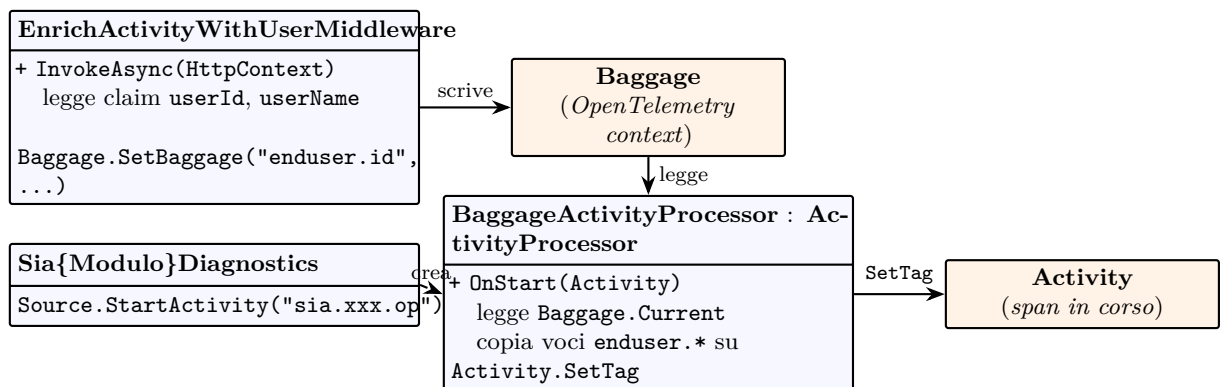


Figura 5.4: Flusso di arricchimento automatico delle `Activity` con l'identità utente. Il *middleware* pubblica `enduser.id` e `enduser.name` nel `Baggage` di `OpenTelemetry`; il `BaggageActivityProcessor` li copia come *tag* su ogni `Activity` creata, comprese quelle prodotte dalle classi `Sia{Modulo}Diagnostics`.

Capitolo 6

Metodologia assistita da agenti AI

Il presente capitolo descrive la metodologia di lavoro adottata durante lo stage, incentrata sull'uso di un agente di codifica basato su un modello linguistico di grandi dimensioni (LLM) come pair engineer strutturato. Vengono illustrati gli ambiti di utilizzo, il workflow Plan → Confirm → Implement, i benefici in termini di tempo e le mitigazioni adottate per i limiti noti.

6.1 Contesto e motivazione

La pianificazione dello stage prevedeva una durata di 300 ore distribuite su 8 settimane, a fronte di un ambito ampio: apprendimento di tecnologie nuove (OpenTelemetry SDK .NET, stack *Grafana* LGTM, PromQL/TraceQL/LogQL, Serilog OTLP, *Playwright*), realizzazione di prototipi, strumentazione di più moduli del framework *Sia.Web* e produzione della relativa documentazione. Per colmare il gap di conoscenza in merito alle tecnologie e lo studio della soluzione *Sia.Web*, si è fatto ricorso all'utilizzo di agenti di intelligenza artificiale. Questo approccio ha permesso di velocizzare le attività ripetitive e di replicare pattern già validati su nuovi moduli. L'introduzione di agenti di codifica, come *Claude Code*, ha reso più efficiente l'introduzione di test automatici e sistemi di telemetria all'interno di un'applicazione di grandi dimensioni, permettendo di creare, e soprattutto propagare, codesti processi di supporto, senza sottrarre eccessive risorse agli sviluppi principali.

6.2 Ambiti di utilizzo

L'agente è stato utilizzato in tre ambiti distinti, corrispondenti a tre momenti differenti dello stage.

6.2.1 Studio delle tecnologie

Nelle prime fasi, dedicate allo studio delle tecnologie, l'agente è stato impiegato per sintetizzare la documentazione ufficiale delle varie tecnologie, e per confrontare le

possibili soluzioni da adottare. Tale attività ha permesso di ridurre in modo sensibile il tempo stimato di *onboarding*, concentrando lo studio sui soli punti critici per il progetto invece che sulla documentazione integrale di ciascuna tecnologia.

6.2.2 Prove di implementazione

Durante la fase di progettazione, l'agente è stato usato per produrre rapidamente varianti di *proof of concept* (PoC) su scelte ancora da validare: che moduli testare, quale fosse il carico computazionale introdotto, l'ambiente di hosting più conveniente (*Docker* o servizi Windows). Avere a disposizione codice compilabile in pochi minuti ha consentito di provare e scartare alternative, invece di decidere a priori sulla base della sola documentazione.

6.2.3 Propagazione della telemetria agli altri moduli

L'ambito in cui l'uso dell'agente si è rivelato più produttivo è stato la propagazione dell'strumentazione ai moduli del framework (*Sia.Base*, *Sia.Auth*, *Sia.Grid*, *Sia.Crud*, ...). Per evitare che la replicazione del pattern di instrumentazione su moduli diversi introducesse incoerenze (naming, cardinalità, attributi di *resource*), sono state definite *Claude skill* dedicate:

- **sia-telemetry-setup**: configurazione del pipeline OTel, *sampler*, *exporter*, *appsettings*;
- **sia-telemetry-traces**: *span*, *tag*, *helper* `TrackQuery`;
- **sia-telemetry-metrics**: metriche, *naming*, cardinalità, pattern `RecordMetricError`;
- **sia-telemetry-logs**: Serilog OTLP, correlazione *log-trace*;
- **sia-telemetry-grafana**: *dashboard* Grafana, query PromQL/TraceQL/LogQL, strumenti MCP.

Ogni *skill* è un contratto procedurale, non un semplice *prompt*, che descrive convenzioni, *anti-pattern* e *template* di codice per uno specifico aspetto. L'agente carica la *skill* appropriata quando riceve un task pertinente, garantendo che moduli instrumentati in momenti diversi rispettino le stesse convenzioni.

6.3 Workflow Agenti AI

Per evitare i rischi tipici della generazione automatica di codice (allucinazioni, *overfitting* al primo caso visto), è stato adottato un *workflow* strutturato in tre fasi.

1. **Plan.** Per ogni task non banale, l'agente produce un file `plan.md` che descrive obiettivo, componenti coinvolti, modifiche pianificate file per file, assunzioni e domande aperte. Nessun codice viene prodotto in questa fase.
2. **Confirm.** Lo sviluppatore revisiona il `plan.md`, può editarlo direttamente, integrarlo o rifiutarlo. L'agente si ferma *esplicitamente* in attesa di approvazione.

3. **Implement.** Solo dopo approvazione l'agente scrive un `task.md`, una *checklist* operativa, nella quale vengono tracciati in ordine, i singoli interventi eseguiti dall'agente.

Il *workflow* produce un artefatto tracciabile per ogni intervento: `plan.md` e `task.md` restano nel repository e fungono da registro decisionale.

6.4 Confronto pianificazione vs esecuzione

L'uso dell'agente ha prodotto un beneficio misurabile soprattutto nelle fasi a forte componente ripetitiva (studio, PoC, propagazione del pattern di instrumentazione), e un beneficio meno marcato in quelle a forte componente decisionale (scelte architetturali, analisi di cardinalità). Nel complesso, il tempo risparmiato nelle fasi assistite è stato riinvestito nelle attività non automatizzabili: revisione semantica del codice prodotto, validazione delle *dashboard* contro Prometheus, affinamento iterativo delle *skill*.

6.5 Limiti e mitigazioni

L'uso di un agente LLM comporta rischi noti che sono stati mitigati in modo sistematico.

Allucinazioni su metriche/label. L'agente tende a inventare nomi di metriche o *label* plausibili ma inesistenti. **Mitigazione:** regola obbligatoria di validazione tramite MCP Grafana (`list_prometheus_metric_names`, `list_prometheus_label_values`) prima di chiudere ogni task dashboard.

Overfitting del prompt. Le *skill* iniziali, scritte a partire dal primo modulo instrumentato (Sia.Base), contenevano riferimenti troppo specifici (nomi di metriche, *tag*, stored procedure). **Mitigazione:** riscrittura delle *skill* in termini generali, con placeholder `{modulo}` e riferimenti ai moduli canonici solo come esempi.

Perdita di contesto su sessioni lunghe. La *context window* dell'LLM è finita; su task lunghi, l'agente perde traccia di decisioni prese in precedenza. **Mitigazione:** memorizzazione persistente tramite MEMORY.md per progetto e per agente, `plan.md` e `task.md` come registro decisionale per ogni task.

Divergenza dei pattern tra moduli. Sviluppatori diversi, o sessioni diverse, possono produrre variazioni del pattern di instrumentazione. **Mitigazione:** centralizzazione dei pattern obbligatori nell'agente `sia-telemetry-developer` (es. counter errori controller via `IControllerDiagnostics` con Adapter, uso obbligatorio della *template variable* `bucket` nelle dashboard).

Per superare gran parte di queste limitazioni e arricchire i processi di ragionamento degli agenti AI con contenuti e contesto aggiuntivi, è stato adottato il protocollo MCP. Considerato inoltre il crescente impatto di questo strumento nell'ecosistema LLM, si è ritenuto opportuno dedicargli una sezione specifica di approfondimento.

6.6 Model Context Protocol

Il **MCP** è il protocollo, introdotto nel 2024, che consente a un agente basato su modelli linguistici di interagire in modo strutturato con servizi esterni, ottenendo dati dinamici e contestuali.

6.6.1 Il problema che MCP risolve

Un modello linguistico di grandi dimensioni (*Large Language Model*, LLM) è, nella sua forma base, un sistema chiuso: conosce ciò che è stato scritto fino alla sua data di addestramento e risponde in linguaggio naturale, ma non può aprire un file, interrogare un database, chiamare un'API o agire nel mondo esterno. Questa limitazione non dipende dall'intelligenza del modello, dipende dall'assenza di un canale strutturato attraverso cui il modello possa ricevere dati aggiornati e restituire azioni concrete.

Prima dell'avvento di **MCP**, ogni integrazione tra un LLM e un sistema esterno richiedeva un connettore ad hoc: codice scritto una volta per quella specifica combinazione di modello e strumento, da riscrivere ogni volta che cambiava uno dei due. Con la proliferazione dei modelli e degli strumenti, il costo di manutenzione di questa matrice di integrazioni punto-a-punto è diventato rapidamente insostenibile. Il **MCP**, introdotto da Anthropic nel novembre 2024 e oggi standard open-source governato dalla Linux Foundation sotto l'egida dell'Agentic AI Foundation, risolve questo problema esattamente come USB-C ha risolto la frammentazione dei connettori fisici: definendo un'interfaccia universale che qualsiasi modello e qualsiasi strumento possono implementare una volta sola per comunicare con l'intero ecosistema.

6.6.2 Architettura

MCP è un protocollo client-server basato su *JSON-RPC 2.0* con connessioni stateful. I ruoli coinvolti sono tre.

Host: L'applicazione che ospita l'LLM e avvia le connessioni **MCP**. Può essere un'interfaccia conversazionale come Claude Desktop, un IDE con assistente integrato, o un servizio backend autonomo. L'host è responsabile del consenso dell'utente e del controllo di accesso: decide quali server **MCP** connettere e con quali permessi.

Client: Il componente interno all'host che gestisce il ciclo di vita della connessione verso uno specifico server **MCP**: negozia le capacità, invia richieste, riceve risposte.

Server MCP: Il servizio che espone dati e funzionalità all'LLM. Può girare in locale (processo figlio, comunicazione via *stdio*) o in remoto (connessione via HTTP con Server-Sent Events). Un server **MCP** può rappresentare un'API esterna, un database, un file system, uno strumento di automazione, o qualsiasi sistema che si voglia rendere accessibile all'agente.

La connessione tra client e server è stateful: prima di qualsiasi scambio applicativo avviene una fase di *handshake* in cui le due parti negoziano le rispettive capacità.

Questo meccanismo garantisce la retrocompatibilità: un client più nuovo può parlare con un server più vecchio senza rompere nulla, e viceversa.

6.6.3 Le primitive del protocollo

MCP definisce tre categorie di primitive che un server può esporre, e tre che un client può offrire al server.

6.6.3.1 Primitive lato server

Resources Dati e contesto che il modello o l'utente possono consultare: file, pagine web, righe di database, output di API. Le risorse sono identificate da URI e possono essere statiche o dinamiche (aggiornate a ogni lettura).

Tools Funzioni che il modello può invocare per compiere azioni nel mondo: creare un file, inviare un'email, eseguire una query SQL, chiamare un endpoint REST. I tool rappresentano esecuzione arbitraria di codice e richiedono consenso esplicito dell'utente prima di essere invocati.

Prompts Template di messaggi e workflow predefiniti che il server mette a disposizione dell'utente o dell'applicazione host, per guidare interazioni ricorrenti senza doverle riscrivere ogni volta.

6.6.3.2 Primitive lato client

Sampling Consente al server di avviare interazioni ricorsive con l'LLM, delegando al client (e quindi all'utente) il controllo su quali prompt vengono effettivamente inviati al modello e cosa il server può vedere delle risposte.

Roots Permette al server di interrogare il client sulle radici del filesystem o degli URI entro cui è autorizzato a operare, definendo i confini del suo spazio di lavoro.

Elicitation Consente al server di richiedere informazioni aggiuntive all'utente in modo strutturato, durante l'esecuzione di un workflow.

6.6.4 Dall'integrazione al microservizio agentic

La portata dell'innovazione si può cogliere quando si analizza il protocollo come, non soltanto un modo più comodo per collegare gli LLM a qualche strumento locale, bensì a **MCP** come **l'infrastruttura per i microservizi agentici**.

Nel modello tradizionale, un microservizio espone una REST API che altri servizi chiamano in modo deterministico: il chiamante sa esattamente quale endpoint invocare, con quali parametri, e interpreta la risposta secondo uno schema fisso. In un'architettura agentic, il chiamante è un LLM che ragiona sul problema, decide autonomamente quali tool invocare e in quale ordine, e compone i risultati in modo flessibile. **MCP** è il contratto che rende possibile questa composizione senza che ogni strumento debba essere scritto appositamente per ogni modello.

Un server **MCP** che espone dati in tempo reale, da una piattaforma di visualizzazione dati come *Grafana*, da un database operativo, da un framework per lo sviluppo

di test (es. *Playwright*), diventa un blocco riutilizzabile dell'ecosistema agentic: qualsiasi agente compatibile con MCP può scoprirlo, negoziare le sue capacità e usarlo, esattamente come un'applicazione scopre e usa un dispositivo USB. La standardizzazione sposta il valore dall'integrazione alla logica applicativa.

6.6.5 Sicurezza

L'ampiezza dei permessi che un server MCP può richiedere, accesso al filesystem, esecuzione di query, chiamate a API esterne, rende la sicurezza un tema centrale che la specifica affronta esplicitamente attraverso quattro principi.

Il primo è il **consenso esplicito dell'utente**: nessun tool può essere invocato senza che l'utente abbia approvato l'azione, e le descrizioni dei tool non devono essere considerate attendibili se provengono da server non verificati (vettore noto di *prompt injection* tramite descrizioni malevole). Il secondo è la **privacy dei dati**: l'host non deve trasmettere dati utente verso server MCP senza consenso esplicito. Il terzo è il **controllo sul sampling**: l'utente deve poter vedere e approvare i prompt che il server vuole inviare al modello. Il quarto è la **separazione dei contesti**: il protocollo limita intenzionalmente la visibilità del server sui prompt dell'utente, riducendo la superficie di attacco per l'estrazione di informazioni confidenziali.

Ricerche indipendenti hanno evidenziato che implementazioni permissive possono essere vulnerabili a *tool poisoning* e *prompt injection* attraverso descrizioni malevole nei server MCP. Per mitigare tali rischi, è consigliabile adottare meccanismi di autorizzazione granulari, in grado di limitare l'accesso ai dati sensibili e di ridurre la superficie di esposizione verso questi server.

6.6.6 Applicazione nel progetto

Nel contesto del progetto *Sia.Core*, MCP è stato impiegato per collegare l'agente di sviluppo direttamente a Grafana, a *Playwright* e a *mssql*, eliminando il passaggio manuale di schemi, nomi di metriche e strutture dati tra l'utente e l'agente.

L'impatto più evidente si è registrato nella generazione delle dashboard: prima dell'integrazione, l'agente produceva widget plausibili ma spesso errati, metriche inesistenti, attributi di filtraggio non presenti nelle serie temporali, tipi di grafico inadatti alla natura del dato. Connettere l'agente a Grafana tramite MCP ha ridotto sensibilmente questi errori, perché il modello poteva interrogare direttamente l'istanza e ricevere come contesto i nomi reali delle metriche, le label disponibili e la struttura delle tracce esistenti in Tempo.

Nonostante il miglioramento, si è resa necessaria una fase di *prompt engineering* mirata a consolidare le linee guida operative dell'agente. In particolare, sono state introdotte direttive esplicite su tre fronti: quali tipi di grafico associare a ciascuna categoria di metrica (es. *time series* per latenze e rate, *stat* per contatori cumulativi, *histogram* per distribuzioni di percentili); quali variabili di template definire a livello di dashboard per abilitare il filtraggio per `service_name`, `sia_repository` e `sia_operation`; come organizzare la griglia dei widget per mantenere leggibilità e coerenza visiva tra moduli diversi.

Un beneficio analogo si è ottenuto con l'integrazione di *Playwright* e *mssql*: in entrambi i casi, l'accesso diretto agli strumenti tramite MCP ha permesso all'agente

di operare su dati reali anziché su assunzioni, riducendo il numero di iterazioni necessarie per ottenere un risultato corretto.

6.7 Considerazioni conclusive

L'utilizzo di un agente AI come supporto allo sviluppo, insieme all'adozione del protocollo [MCP](#), ha contribuito in modo significativo al miglioramento della qualità del prodotto, alla riduzione dei tempi di sviluppo e al superamento delle iniziali lacune tecniche.

Tuttavia, il ruolo dell'agente AI non deve essere interpretato come sostitutivo di quello dello sviluppatore. Il lavoro eseguito dall'agente, che fosse codice prodotto, scelte architetturali o propagazione degli sviluppi, ha spesso richiesto interventi di indirizzamento, correzione e validazione da parte dello sviluppatore risultando quindi un processo collaborativo piuttosto che autonomo.

Capitolo 7

Codifica

Il presente capitolo illustra l'organizzazione del codice prodotto e i dettagli implementativi della soluzione di telemetria, riprendendo le scelte progettuali descritte nel capitolo 5 e mostrando come si traducono nella struttura effettiva del framework `Sia.Server.Framework`.

7.1 Organizzazione del codice

Il codice di telemetria è distribuito su due livelli del framework:

- un livello **condiviso** (`Application.Shared.Telemetry` e `BaseApplicationBuilder`), che configura il pipeline `OTel`, registra *sampler*, *processor* e *exporter*, e definisce le convenzioni comuni (attributi di *resource*, *middleware* di arricchimento utente);
- un livello **per-modulo** (`Sia.{Modulo}/Diagnostics/`), che contiene la classe statica `Sia{Modulo}Diagnostics` con `ActivitySource`, `Meter`, `Counter`, `Histogram`, e la dashboard Grafana `{nome}.dashboard.json` versionata insieme al codice.

Questa suddivisione garantisce che un nuovo modulo, per integrare la telemetria, debba solo creare la propria classe `Diagnostics` e registrare il proprio `Meter` e `ActivitySource` nel pipeline già costruito, senza conoscere i dettagli del *sampler* o degli *exporter*.

7.2 Pipeline OpenTelemetry

Il pipeline è configurato in `BaseApplicationBuilder.ConfigureBuilder` tramite l'[API](#) `AddOpenTelemetry()` dell'estensione ufficiale. I pacchetti NuGet utilizzati sono:

- `OpenTelemetry` e `OpenTelemetry.Extensions.Hosting` per l'integrazione con il *generic host* di .NET;
- `OpenTelemetry.Instrumentation.AspNetCore` e `OpenTelemetry.Instrumentation.Http` per le *trace* automatiche delle richieste HTTP in ingresso e in uscita;

- `OpenTelemetry.Instrumentation.SqlClient` per le *trace* automatiche delle query su SQL Server;
- `OpenTelemetry.Exporter.OpenTelemetryProtocol` per l'esportazione OTLP verso il *Collector*.

La configurazione dichiara nell'ordine: il `ResourceBuilder` con gli attributi comuni (`service.name`, `sia.product_type`, `sia.customer_name`), il provider di *trace* (`WithTracing`) con *sampler* e *processor* custom, il provider di metriche (`WithMetrics`) con il filtro exemplar `TraceBased`, e il provider di *log* (`WithLogging`) integrato con Serilog.

7.3 Classi `Sia{Modulo}Diagnostics`

Ciascun modulo strumentato espone una classe statica `Sia{Modulo}Diagnostics` nel *namespace* `Sia.{Modulo}.Diagnostics`. La classe definisce come membri statici *readonly*:

- l'`ActivitySource` del modulo, con nome `Sia.{Modulo}` e versione;
- il `Meter` del modulo, con lo stesso nome e versione;
- i `Counter/Histogram/UpDownCounter` custom, con *naming* secondo la convenzione `sia.{modulo}.{operazione}.{segnale}`;
- per i moduli con controller, il `Counter ControllerErrors` (`sia.{modulo}.controller.errors`) e il singleton `AsControllerDiagnostics` dell'Adapter verso `IControllerDiagnostics`.

Nel codice applicativo, l'istrumentazione manuale di un'operazione avviene con:

```
Sia{Modulo}Diagnostics.ActivitySource.StartActivity("sia.{modulo}.operazione");
```

seguito da `activity?.SetTag(...)` per gli attributi specifici. Per le durate si usa un *helper* `TrackQuery` che combina in un unico blocco la creazione della *span*, la misurazione del tempo e la registrazione dell'`Histogram` corrispondente.

7.4 Pattern `RecordMetricError` con Adapter

L'implementazione del pattern descritto nella sezione 5.5 è la stessa in ogni modulo con controller. Il framework fornisce:

- l'interfaccia `Sia.Base.Diagnostics.IControllerDiagnostics`, con il contratto `Counter<long> ControllerErrors { get; };`
- il metodo `protected static BaseApiController.RecordMetricError(IControllerDiagnostics Exception, string operation)`, che effettua l'*unwrap* di `SiaLoggerException.OriginalException` e applica i *tag* `exception.type` e `sia.operation`.

Nel modulo (esempio riferito a `Sia.Devextreme.Crud`) la classe `Diagnostics` si presenta, nei suoi elementi essenziali, come segue:

```

public static class SiaDevextremeCrudDiagnostics
{
    public static readonly Meter Meter = new("Sia.Devextreme.Crud", "1.0.0");

    public static readonly Counter<long> ControllerErrors =
        Meter.CreateCounter<long>("sia.devextreme.crud.controller.errors");

    public static readonly IControllerDiagnostics AsControllerDiagnostics =
        new ControllerDiagnosticsAdapter();

    private sealed class ControllerDiagnosticsAdapter : IControllerDiagnostics
    {
        public Counter<long> ControllerErrors =>
            SiaDevextremeCrudDiagnostics.ControllerErrors;
    }
}

```

Nei controller, ogni `catch` richiama il metodo condiviso:

```

catch (Exception ex)
{
    RecordMetricError(
        SiaDevextremeCrudDiagnostics.AsControllerDiagnostics,
        ex,
        nameof(GetData));
    throw;
}

```

7.5 *Middleware* di arricchimento utente

Il *middleware* `Application.Shared.Telemetry.EnrichActivityWithUserMiddleware` è registrato in `BaseApplicationBuilder` subito dopo `app.UseAuthentication()`. La sua responsabilità è leggere le *claim* `userId` e `userName` dal `HttpContext.User` e pubblicarle nel `Baggage` di [OTel](#). Un `ActivityProcessor` custom (`BaggageActivityProcessor`) intercetta poi l'hook `OnStart` di ogni `Activity` e copia ogni *entry* di *baggage* con prefisso `enduser.` come *tag* sull'`Activity`.

7.6 Configurazione degli *exemplars*

Nel provider `Meter` del pipeline è presente l'istruzione `SetExemplarFilter(ExemplarFilterType.TraceBased)`. Questa configurazione istruisce [OTel](#) ad allegare un *exemplar* a un campione di metrica solo quando esiste una trace *sampled* nel contesto corrente. L'*exemplar* riporta il `TraceId` (e il `SpanId`) del campione, permettendo in Grafana la navigazione diretta dalla metrica alla trace.

7.7 Log Serilog con *sink* OTLP

Il *logger* è configurato in `Sia.Logger` tramite Serilog. Il *sink* OTLP (`Serilog.Sinks.OpenTelemetry`) è registrato in sostituzione dei *sink* precedenti, mantenendo il formato strutturato dei *log*. Serilog legge il contesto di `Activity.Current` e arricchisce automaticamente ogni evento con `TraceId` e `SpanId`, soddisfacendo il requisito RFN-4 di correlazione *log-trace*. Il livello minimo di *log* è configurabile per ambiente tramite *appsettings*.

7.8 Dashboard Grafana e versionamento

Ogni modulo instrumentato espone una dashboard Grafana nella cartella `Sia.{Modulo}/Diagnostics/{nome}.dashboard.json`. Il file JSON è versionato nel repository insieme al codice che instrumenta, in modo da garantire il sincronismo tra le modifiche di istrumentazione e i pannelli di visualizzazione. Ogni dashboard definisce:

- le *template variable* `service_name`, `enduser_id`, `enduser_name` e `bucket`;
- pannelli di latenza con percentili *p50*, *p95*, *p99* calcolati via `histogram_quantile` (Figura 7.1);
- un pannello *stat* “Operazioni ultimo `#{bucket}`” basato sull’increase del *counter* principale (Figura 7.2);
- pannelli di errori con *break-down* per `exception.type` (Figura 7.3);
- un pannello *logs* su Loki filtrato per `service.name` (Figura 7.3).

La riferimento canonica è la dashboard `sia-base.dashboard.json` del modulo `Sia.Base`, replicata con le dovute personalizzazioni in ogni modulo successivo.

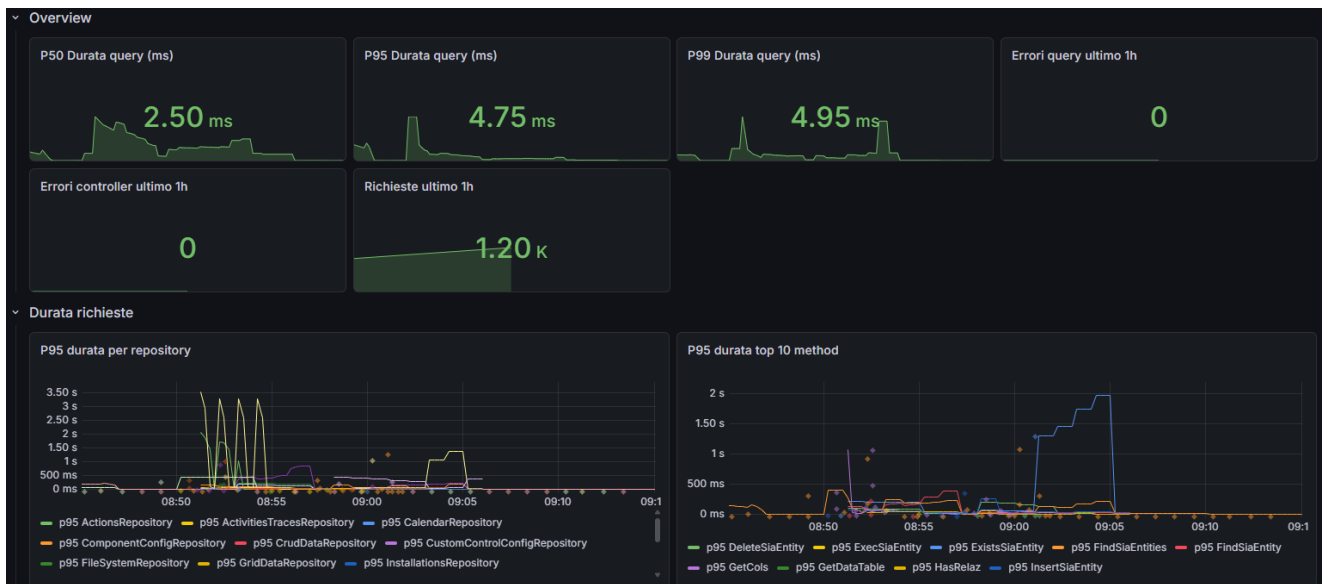


Figura 7.1: Overview e durata richieste della dashboard di *Sia.Base*

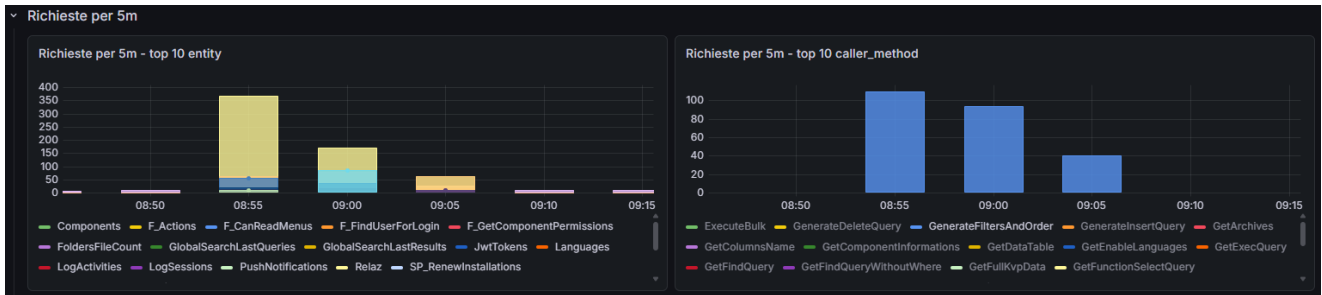


Figura 7.2: Sezione richieste per $\{\text{bucket}\}$ della dashboard *Sia.Base*

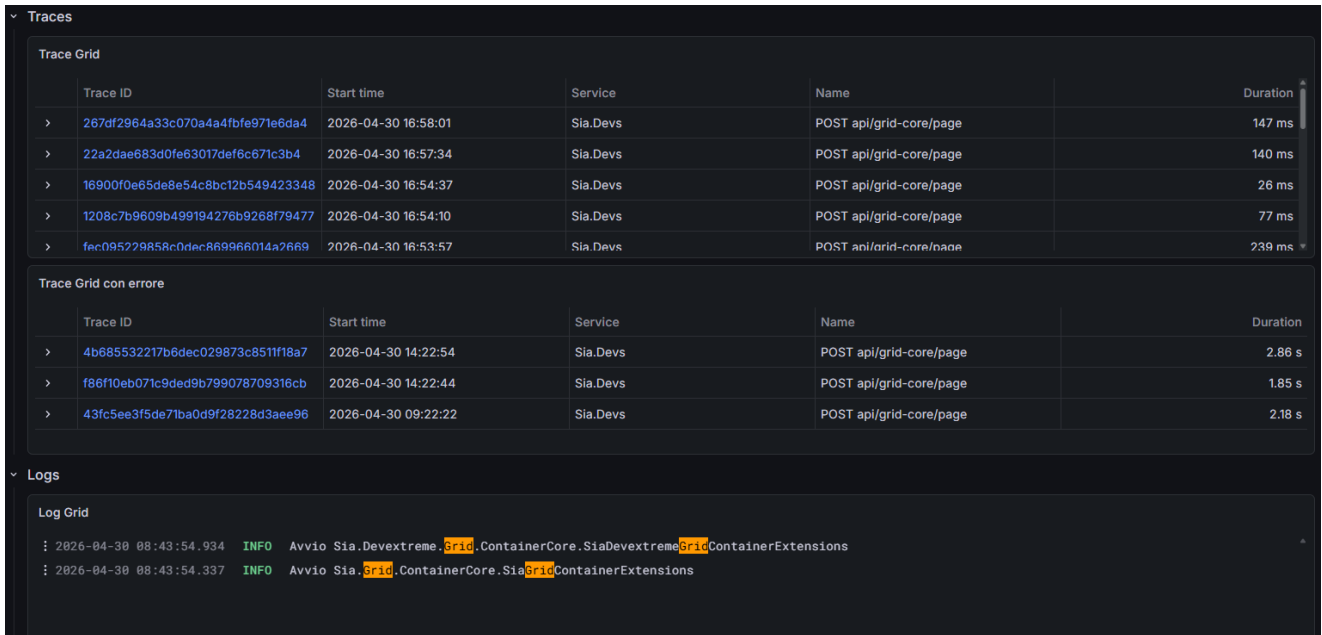


Figura 7.3: Sezione trace e log della dashboard di *Sia.Grid*

7.9 Telemetria delle performance hardware

7.9.1 Contesto e obiettivo

Il modulo `Sia.HealthChecks.Servers` è parte del framework *Sia.Server.Framework* ed è responsabile della raccolta periodica di indicatori sullo stato del server che ospita l'applicazione, con successiva notifica ad un *manager* centrale delle installazioni. Prima dell'intervento di telemetria, il modulo esponeva già i dati tramite:

- cinque implementazioni di `IHealthCheck` (`CpuUsageHealthCheck`, `CpuTemperatureHealthCheck`, `RamHealthCheck`, `DiskHealthCheck`, `SqlHealthCheck`), invocate dall'endpoint `/health` di ASP.NET Core;
- un `BackgroundService` (`HealthCheckServerHostedService`) che, ad intervallo configurabile (`HealthNotifyDelayMinutes`), aggrega i risultati e invia il report al *manager* tramite un `HttpClient` dedicato (`HealthCheckServerToServerApi`).

7.9.2 Scelte progettuali

Il modulo presenta due caratteristiche che lo distinguono dagli altri moduli instrumentati.

La prima è l'assenza di un controller: gli `IHealthCheck` sono invocati dall'infrastruttura ASP.NET Core, non da endpoint applicativi. Di conseguenza `SiaHealthChecksServersDiagnostics` non implementa `IControllerDiagnostics` e non espone il counter `ControllerErrors`.

La seconda è la natura *istantanea* dei dati raccolti. CPU%, temperatura, GB di RAM liberi e spazio su disco non sono grandezze cumulative: l'uso di `Counter` o `Histogram` sarebbe improprio. Lo strumento corretto è l'`ObservableGauge`, la cui callback viene invocata on-demand al momento dello *scrape* di Prometheus. Poiché le letture hardware avvengono solo quando l'endpoint `/health` viene interrogato, le gauge leggono un valore **in cache**, aggiornato dagli `IHealthCheck` alla loro esecuzione, evitando di esporre dati stantii.

7.9.3 Segnali prodotti

7.9.3.1 Trace

Il modulo produce tre span custom, organizzate gerarchicamente.

Tabella 7.1: Span prodotte dal modulo `Sia.HealthChecks.Servers`

Span	Contesto	Tag principali
<code>...cycle</code>	Root span di ogni iterazione del <code>HostedService</code>	<code>sia.enabled</code> , <code>sia.delay_minutes</code> , <code>sia.success</code> , <code>exception.type</code>
<code>...check</code>	Child span per ogni <code>IHealthCheck</code> invocato	<code>sia.check</code> , <code>sia.status</code> , <code>sia.caller_method</code>
<code>...send</code>	Span attorno alla chiamata di invio al <i>manager</i>	<code>sia.server_id</code> , <code>sia.result</code> , <code>exception.type</code>

Le trace HTTP generate automaticamente da `HttpClient` restano figlie di `...send`, permettendo di ispezionare nella stessa trace sia la logica applicativa sia il dettaglio del POST verso il manager.

7.9.3.2 Metriche

Accanto alle metriche cumulative standard (`cycle`, `check`, `send`), il modulo espone sei `ObservableGauge` per i valori hardware istantanei.

I tag hanno domini piccoli e chiusi (`sia.check` $\in \{cpu, cpu_temperature, ram, disk, sql\}$, `sia.status` $\in \{Healthy, Unhealthy\}$), mantenendo la cardinalità sotto controllo indipendentemente dal numero di installazioni monitorate.

7.9.4 Dashboard Grafana

La dashboard segue lo schema standard del progetto, con l'aggiunta di una riga *Risorse* dedicata alle gauge hardware: andamento di CPU% e temperatura nel

Tabella 7.2: ObservableGauge del modulo `Sia.HealthChecks.Servers`

Metrica	Unità	Descrizione
<code>...cpu.usage</code>	%	Percentuale di utilizzo della CPU al momento dell'ultima lettura
<code>...cpu.temperature</code>	°C	Temperatura del processore rilevata dai sensori hardware
<code>...ram.available</code>	GByte	Memoria RAM libera disponibile per i processi
<code>...ram.total</code>	GByte	Memoria RAM fisica totale installata nel server
<code>...disk.free</code>	GByte	Spazio libero sul disco, distinto per <code>sia.drive</code>
<code>...disk.total</code>	GByte	Capacità totale del disco, distinta per <code>sia.drive</code>

tempo, RAM disponibile confrontata a soglia, spazio disco libero per drive e una *state timeline* delle connessioni SQL che visualizza a colpo d'occhio gli intervalli in cui una connessione è risultata *down*. I pannelli trace e log consentono, a partire da un picco anomalo su qualsiasi metrica hardware, di navigare alla trace del ciclo di notifica corrispondente e ai log emessi in quell'iterazione, chiudendo il percorso diagnostico *metrica* → *trace* → *log*.

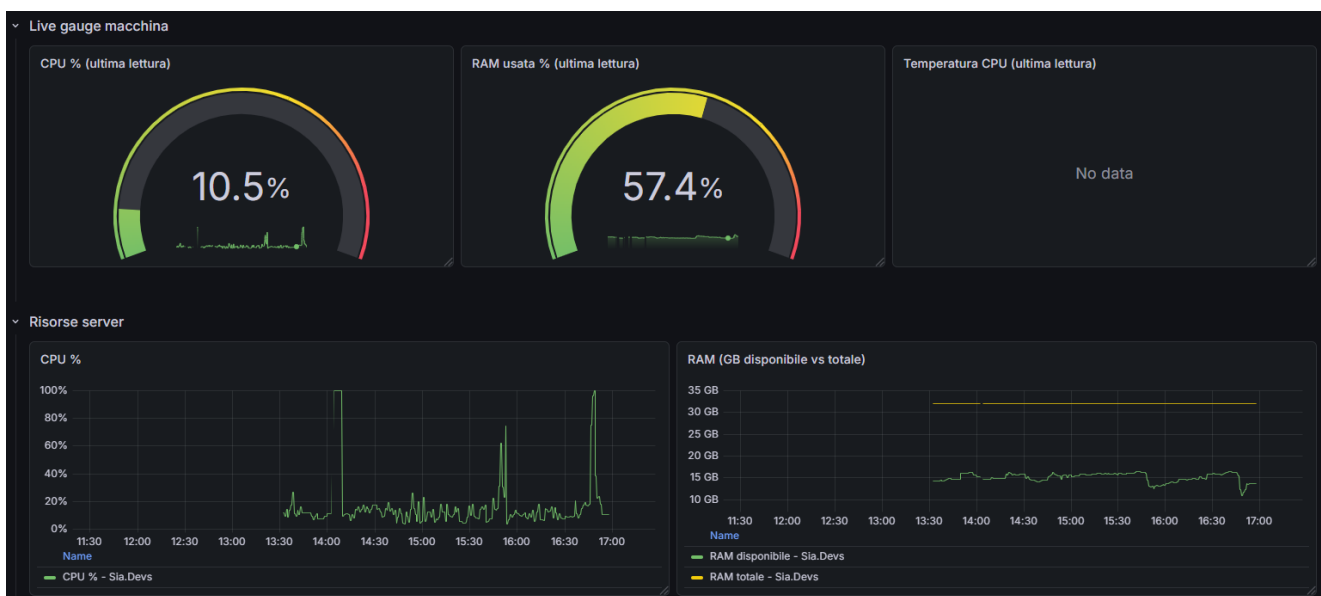


Figura 7.4: Sezione risorse della dashboard di telemetria hardware

Capitolo 8

Conclusioni

Il presente capitolo tira le somme del lavoro svolto durante lo stage, confrontando la pianificazione con l'esecuzione reale, valutando il raggiungimento degli obiettivi, sintetizzando le conoscenze acquisite e proponendo possibili evoluzioni future.

8.1 Consuntivo finale

Il progetto si è sviluppato sulle 8 settimane pianificate, per un totale di 300 ore. Il monte ore è stato complessivamente rispettato, con una redistribuzione interna: le fasi di studio (settimane 1–3) e di prototipazione (settimana 4) hanno richiesto un numero di ore inferiore al preventivato grazie all'uso dell'agente AI come *pair engineer*, mentre le fasi di propagazione dell'instrumentazione ad altri moduli e di costruzione delle dashboard Grafana sono state estese rispetto alla pianificazione originaria, assorbendo le ore risparmiate nella prima parte.

8.2 Raggiungimento degli obiettivi

Gli obiettivi dichiarati nella sezione “Obiettivi” del capitolo 2 sono stati raggiunti:

- l'analisi dei requisiti di osservabilità e la selezione delle tecnologie hanno portato all'adozione dello stack [OTel](#) + *Grafana LGTM*, motivata dai criteri descritti nel capitolo 4;
- l'architettura di telemetria è stata progettata e integrata con il framework *.NET*, centralizzando la configurazione in `BaseApplicationBuilder` (capitolo 5);
- le strategie di campionamento sono state definite a due livelli (*sampler* globale e *processor* per-source), con ratio configurabili per ambiente;
- gli strumenti di istrumentazione sono stati implementati per i moduli *Sia.Auth*, *Sia.Base* e *Sia.Grid*, e propagati successivamente ad altri moduli del framework tramite le *Claude skill* dedicate;
- la piattaforma di visualizzazione Grafana è stata configurata con i relativi *backend* di *storage* (Tempo, Loki, Prometheus) e con le dashboard versionate nel repository.

I requisiti funzionali RFN-1..RFN-11 e non funzionali sono stati soddisfatti nella loro versione *obbligatoria*. In particolare, i requisiti aggiuntivi RFN-10 (arricchimento automatico con `enduser.*`) e RFN-11 (*exemplars* metrica $\rightarrow$ trace) sono stati implementati come descritto nelle sezioni ?? e ??.

8.3 Conoscenze acquisite

Lo stage ha portato all'acquisizione di competenze in più ambiti:

- standard OTel e modello dei segnali (*trace*, metriche, *log*, *exemplars*);
- stack *Grafana* (Tempo, Prometheus, Loki), con i rispettivi linguaggi di query PromQL, TraceQL, LogQL;
- libreria Serilog e *sink* OTLP, con correlazione *log-trace*;
- applicazione dei *design pattern* Adapter e Factory a un contesto reale di framework condiviso;
- metodologie di lavoro assistito da agenti AI, *prompt* e *skill engineering*, uso del MCP per integrare l'agente con servizi runtime.

8.4 Retention e gestione dei dati di telemetria

Un aspetto operativo affrontato durante lo stage è la *retention* dei dati di telemetria. Ogni segnale ha caratteristiche di storage differenti:

- le *trace* su Tempo sono voluminose per singolo campione ma a bassa densità (sono campionate); la *retention* suggerita è *medio-breve* (7–14 giorni), sufficiente per la diagnosi di incidenti recenti;
- le metriche su Prometheus (o Mimir) sono aggregate e compatte: la *retention* può estendersi a 90 giorni o più, permettendo il confronto di tendenze nel tempo;
- i *log* su Loki sono voluminosi ma indicizzati per *label*: la *retention* tipica è di 30 giorni, con possibilità di archiviazione a lungo termine in storage a basso costo.

I tempi concreti di *retention* sono configurati in funzione delle esigenze di indagine e del costo di *storage* accettabile per il singolo cliente.

8.5 Valutazione personale e sviluppi futuri

Dal punto di vista personale, lo stage ha permesso di lavorare a stretto contatto con un *framework* enterprise reale, cimentandosi sia con la progettazione (scelte di cardinalità, *sampling*, correlazione *cross-signal*) sia con la meta-programmazione del processo di sviluppo tramite *Claude skill*. La collaborazione con il *team* aziendale e il

confronto continuo sulle scelte architetturali sono stati parte integrante del percorso formativo.

Tra gli sviluppi futuri più promettenti si collocano:

- introduzione del *tail-based sampling* tramite *Collector*, che permetterebbe di conservare integralmente le tracce di richieste lente o in errore anche con un *ratio* globale basso;
- definizione di metriche *RED* (Rate, Errors, Duration) e *USE* (Utilization, Saturation, Errors) in forma automatizzata per ogni nuovo modulo;
- definizione di *Service Level Objective* (SLO) e *alerting* basato sul *burn-rate* dell'*error budget*;
- integrazione di *continuous profiling* tramite *Pyroscope*, già presente nello stack Grafana, per correlare tracce e profili di CPU/memoria;
- estensione del pattern di *skill engineering* ad altri ambiti del framework, oltre alla telemetria (es. generazione di *CRUD*, di *controller*, di *dashboard* di *business*).

Appendice A

Appendice A

Citazione

Autore della citazione

Acronimi e abbreviazioni

AI Artificial Intelligence. 9–11, 54

API Application Program Interface. 9, 10, 18, 19, 22, 26, 42, 54

CRM Customer Relationship Management. 1, 54

CRUD Create, Read, Update, Delete. ix, 27, 54

ERP Enterprise Resource Planning. 1, 54

LLM Large Language Model. 7, 8, 12, 13, 20, 35, 37, 54

MCP Model Context Protocol. 3, 4, 20, 36–41, 50, 54

OTel OpenTelemetry. 5, 6, 12, 16, 22, 24, 25, 28, 31, 32, 42, 44, 49, 50

PoC Proof of Concept. 17, 36, 37, 55

RFID Radio-Frequency Identification. 1, 55

SDK Software Development Kit. 55

SPA Single Page Application. 18, 55

UML Unified Modeling Language. 12, 32, 55

Glossario

AI con *AI, Artificial Intelligence* si indica un insieme di tecniche e metodi informatici che consentono a un sistema di simulare capacità tipiche dell'intelligenza umana, quali apprendimento, ragionamento, percezione e risoluzione di problemi. 53

API in informatica con il termine *Application Programming Interface API* (ing. interfaccia di programmazione di un'applicazione) si indica ogni insieme di procedure disponibili al programmatore, di solito raggruppate a formare un set di strumenti specifici per l'espletamento di un determinato compito all'interno di un certo programma. La finalità è ottenere un'astrazione, di solito tra l'hardware e il programmatore o tra software a basso e quello ad alto livello semplificando così il lavoro di programmazione. 53

CRM con *CRM, Customer Relationship Management* si indica un sistema o insieme di strumenti software utilizzati per gestire le relazioni e le interazioni con clienti attuali e potenziali. Un CRM consente di organizzare e analizzare informazioni relative ai clienti, supportare le attività di vendita, marketing e assistenza, e migliorare la qualità del servizio e la fidelizzazione. 53

CRUD con *CRUD, Create, Read, Update, Delete* si indica l'insieme delle quattro operazioni fondamentali per la gestione dei dati in un sistema informatico. Rappresenta le azioni di creazione, lettura, aggiornamento ed eliminazione di entità all'interno di un database o di un'applicazione. Queste operazioni costituiscono la base per la maggior parte delle funzionalità di persistenza e manipolazione dei dati.. 53

ERP con *ERP, Enterprise Resource Planning* si indica un sistema informativo aziendale integrato utilizzato per gestire e automatizzare i principali processi operativi di un'organizzazione, quali contabilità, gestione delle risorse umane, produzione, logistica e vendite. 53

LLM con *LLM, Large Language Model* si indica un modello di intelligenza artificiale basato su reti neurali di grandi dimensioni, addestrato su ampie quantità di dati testuali, in grado di comprendere e generare linguaggio naturale. 53

MCP *MCP, Model Context Protocol* è un protocollo che consente a un agente basato su modelli di intelligenza artificiale di interagire con sistemi e servizi esterni in modo strutturato, permettendo l'accesso a dati dinamici e contestuali. MCP abilita l'integrazione tra modelli linguistici e strumenti operativi, migliorando

la capacità decisionale dell'agente e riducendo la dipendenza da informazioni statiche. [53](#)

PoC con *PoC, Proof of Concept* si indica un prototipo o implementazione preliminare sviluppata per verificare la fattibilità di una soluzione tecnica o di un'idea progettuale. Un PoC ha lo scopo di validare determinati aspetti, come prestazioni, integrazione o corretto funzionamento, prima di procedere con uno sviluppo completo. [53](#)

RFID con *RFID, Radio-Frequency Identification* si indica una tecnologia che utilizza onde radio per identificare e tracciare oggetti, persone o animali attraverso dispositivi chiamati tag RFID. Questi tag contengono informazioni che possono essere lette a distanza da appositi lettori, senza necessità di contatto diretto o visibilità, rendendo la tecnologia particolarmente utile in ambiti come logistica, tracciamento e gestione degli inventari. [53](#)

SCRUM metodologia agile per la gestione e lo sviluppo di progetti, basata su un approccio iterativo e incrementale. Il lavoro viene suddiviso in cicli temporali chiamati Sprint, al termine dei quali viene prodotto un incremento funzionante del sistema. Scrum definisce ruoli (come Scrum Master e Product Owner), eventi (Sprint, Daily Scrum, Sprint Review e Retrospective) e artefatti (Product Backlog e Sprint Backlog) per favorire collaborazione, adattabilità e miglioramento continuo del processo di sviluppo. [7](#)

SDK con *SDK, Software Development Kit* si indica insieme di strumenti, librerie, documentazione e API messi a disposizione per facilitare lo sviluppo di applicazioni su una determinata piattaforma o tecnologia. Un SDK fornisce componenti riutilizzabili e funzionalità predefinite che permettono agli sviluppatori di integrare rapidamente servizi specifici, riducendo tempi e complessità di sviluppo.. [53](#)

SPA con *SPA, Single Page Application* si indica un'applicazione web costituita da una singola pagina HTML che viene aggiornata dinamicamente senza ricaricare completamente il contenuto. Le SPA migliorano l'esperienza utente rendendo la navigazione più fluida e veloce, grazie al caricamento asincrono dei dati e alla gestione lato client della logica applicativa. [53](#)

UML in ingegneria del software *UML, Unified Modeling Language* (ing. linguaggio di modellazione unificato) è un linguaggio di modellazione e specifica basato sul paradigma object-oriented. L'*UML* svolge un'importantissima funzione di "lingua franca" nella comunità della progettazione e programmazione a oggetti. Gran parte della letteratura di settore usa tale linguaggio per descrivere soluzioni analitiche e progettuali in modo sintetico e comprensibile a un vasto pubblico. [53](#)

Bibliografia

Riferimenti bibliografici

Gamma, Erich et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994 (cit. a p. 29).

James P. Womack, Daniel T. Jones. *Lean Thinking, Second Editon*. Simon & Schuster, Inc., 2010.

Siti web consultati

.NET. URL: <https://dotnet.microsoft.com/it-it/>.

Angular. URL: <https://angular.dev/overview>.

C#. URL: <https://dotnet.microsoft.com/it-it/languages/csharp>.

Claude Code. URL: <https://www.anthropic.com/claude-code>.

Grafana. URL: <https://grafana.com/>.

Grafana Loki Documentation. URL: <https://grafana.com/docs/loki/latest/>.

Grafana Mimir Documentation. URL: <https://grafana.com/docs/mimir/latest/>.

Grafana Tempo Documentation. URL: <https://grafana.com/docs/tempo/latest/>.

Manifesto Agile. URL: <http://agilemanifesto.org/iso/it/>.

Model Context Protocol Specification. URL: <https://modelcontextprotocol.io/>.

OpenTelemetry. URL: <https://opentelemetry.io/>.

OpenTelemetry Metrics Data Model — Exemplars. URL: <https://opentelemetry.io/docs/specs/otel/metrics/data-model/#exemplars>.

OpenTelemetry Specification. URL: <https://opentelemetry.io/docs/specs/otel/>.

Serilog Sinks OpenTelemetry. URL: <https://github.com/serilog/serilog-sinks-opentelemetry>.

Spec-Kit. URL: <https://github.com/spec-kit/>.

TypeScript. URL: <https://www.typescriptlang.org/>.